

Article

CCFD: Efficient Credit Card Fraud Detection Using Meta-Heuristic Techniques and Machine Learning Algorithms

Diana T. Mosa ^{1,2} , Shaymaa E. Sorour ^{3,4,*} , Amr A. Abohany ²  and Fahima A. Maghraby ⁵ 

- ¹ Department of Cyber Security, College of Engineering and Information Technology, Buraydah Private Colleges, Buraydah 51418, Saudi Arabia; Diana.Tharwat@bpc.edu.sa
- ² Faculty of Computers and Information, Kafrelsheikh University, Kafrelsheikh 33516, Egypt; cio_kfs@kfs.edu.eg
- ³ Department of Management Information Systems, School of Business, King Faisal University, Alhufuf 31982, Saudi Arabia
- ⁴ Faculty of Specific Education, Kafrelsheikh University, Kafrelsheikh 33511, Egypt
- ⁵ College of Computing and Information Technology, Arab Academy for Science, Technology, and Maritime Transport, Cairo 2033, Egypt; fahima@aast.edu
- * Correspondence: ssorour@kfu.edu.sa

Abstract: This study addresses the critical challenge of data imbalance in credit card fraud detection (CCFD), a significant impediment to accurate and reliable fraud prediction models. Fraud detection (FD) is a complex problem due to the constantly evolving tactics of fraudsters and the rarity of fraudulent transactions compared to legitimate ones. Efficiently detecting fraud is crucial to minimize financial losses and ensure secure transactions. By developing a framework that transitions from imbalanced to balanced data, the research enhances the performance and reliability of FD mechanisms. The strategic application of Meta-heuristic optimization (MHO) techniques was accomplished by analyzing a dataset from Kaggle's CCF benchmark datasets, which included data from European credit-cardholders. They evaluated their capability to pinpoint the smallest, most relevant set of features, analyzing their impact on prediction accuracy, fitness values, number of selected features, and computational time. The study evaluates the effectiveness of 15 MHO techniques, utilizing 9 transfer functions (TFs) that identify the most relevant subset of features for fraud prediction. Two machine learning (ML) classifiers, random forest (RF) and support vector machine (SVM), are used to evaluate the impact of the chosen features on predictive accuracy. The result indicated a substantial improvement in model efficiency, achieving a classification accuracy of up to 97% and reducing the feature size by up to 90%. In addition, it underscored the critical role of feature selection in optimizing fraud detection systems (FDSs) and adapting to the challenges posed by data imbalance. Additionally, this research highlights how machine learning continues to evolve, revolutionizing FDSs with innovative solutions that deliver significantly enhanced capabilities.

Keywords: Credit Card Fraud (CCF); Feature Selection (FS); Meta-heuristic Optimization (MHO); Transfer Functions (TFs)

MSC: 37M05; 37M10



Citation: Mosa, D.T.; Sorour, S.E.; Abohany, A.A.; Maghraby, F.A. CCFD: Efficient Credit Card Fraud Detection Using Meta-Heuristic Techniques and Machine Learning Algorithms. *Mathematics* **2024**, *12*, 2250. <https://doi.org/10.3390/math12142250>

Academic Editors: Yu Xue, Chunlin He and Ferrante Neri

Received: 8 June 2024

Revised: 16 July 2024

Accepted: 17 July 2024

Published: 19 July 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Electronic commerce (E-commerce) is the revolutionary method of buying and selling goods and services using the internet. This evolution in traditional business models has made it possible for consumers to shop from the comfort of their homes [1,2]. E-commerce platforms offer several advantages, such as speeding up procurement processes, reducing costs, improving customer convenience, enabling easy comparison of products and prices, quickly adjusting to market changes, and offering various payment options. These features have significantly supported economic stability, primar-

ily when government restrictions and stay-at-home orders were enforced during pandemics like COVID-19 [3,4]. In 2020, eMarketer reports indicated that sales worldwide grew by 27.6% and were forecasted to grow by an additional 14.3% in 2021, approaching 5 trillion (<https://www.insiderintelligence.com/content/worldwide-ecommerce-will-approach-5-trillion-this-year> (accessed on 16 July 2024)).

The 2008–2009 financial crisis saw a substantial surge in e-commerce revenue growth from 15% to 25% annually. However, due to the economic downturn, it subsequently plummeted to around 3% in 2009. Post-2009, the development of E-commerce bounced back, climbing over 10% each year, a figure significantly exceeding that of overall sales. In 2020, the COVID-19 pandemic, despite its negative impact on digital travel sales, sparked a notable rise in revenue from E-commerce. This was driven by a shift in consumer behavior towards online shopping, a trend that has persisted beyond the pandemic [5].

The ongoing COVID-19 pandemic has resulted in a surge in demand for goods, including necessities. This trend has also led to an increased use of digital payment options, which has increased financial fraud. As e-commerce continues to evolve, businesses of all sizes have started accepting credit card (CC) payments. However, this uptick in CC use, especially for online purchases, has given fraudsters avenues to exploit and steal consumer CC details [6].

Financial fraud significantly impacts academic, commercial, and regulatory spheres, presenting a significant challenge for service providers and their clientele. This critical issue permeates the financial sector, affecting daily economic transactions worldwide. It involves the unauthorized use of assets or funds for personal gain, undermining confidence in financial institutions and leading to higher living costs. Financial fraud encompasses various activities, including falsifying financial statements, telecommunications fraud, and deceptive insurance practices misconduct in the markets. The widespread consequences of fraudulent actions have led to significant disruptions in the global financial system, hastening the shift towards digital financial services and introducing new challenges [7,8].

Between 2000 and 2015, there was a substantial surge in the amount of money lost to fraud related to credit and debit cards. Carta et al. [9] emphasize that while illegal transactions and counterfeit CCs make up from 10 to 15% of all frauds, they account for an overwhelming 75–80% of financial fraud damages. This has increased private and public investments in developing more advanced systems to detect fraudulent activities. The significant monetary transactions in E-commerce make it a prime target for fraud, leading to potential substantial economic losses. The report from Juniper Research clearly shows an alarming increase in fraud-related financial loss, soaring from 17.5 billion in 2020 to a staggering 20 billion in 2021. This underscores the urgent imperative for financial institutions to bolster their CCFD measures without delay [10].

Fraud is an unauthorized action marked by deceit. Credit card fraud (CCF) involves illegally obtaining a cardholder's details through phone calls, text messages, or online hacking to carry out unauthorized transactions. These fraud acts typically involve using software controlled by the fraudster. The process of detecting CCF starts when a customer purchases using their information. This transaction must be verified to ensure its authenticity [10–12].

Statistics reveal that in the fourth quarter of 2020, Visa and Mastercard distributed 2287 million CCs globally, as shown in Figure 1a,b. Visa was responsible for issuing 1131 million cards, while Mastercard issued 1156 million. This data highlights the growing ease and popularity of card-based transactions among consumers. However, it also indicates a potential risk, as this significant volume of transactions attracts fraudsters looking to exploit card users [13].

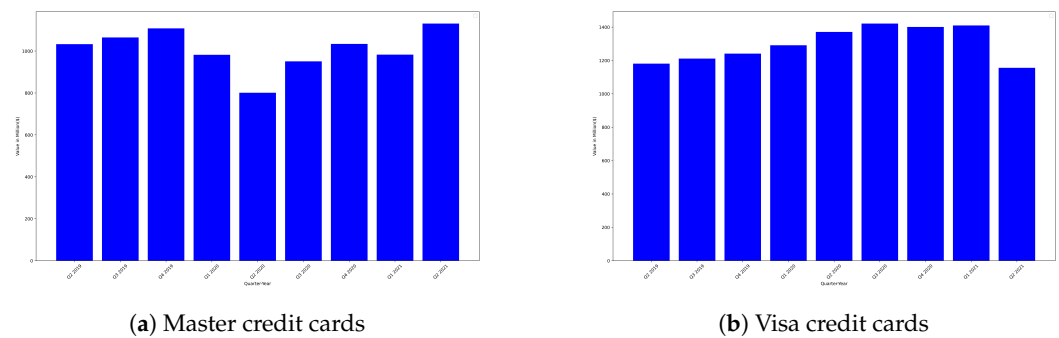


Figure 1. Amount of Master and Visa CCs issued worldwide [13].

Research efforts are centered on creating detection systems that utilize ML, data mining (DM), and deep learning (DL) approaches. These systems analyze transactions to distinguish between legitimate and deceptive transactions. As fraudulent transactions increasingly mimic legitimate ones, the challenge of detecting CCF grows. This necessitates the adoption of more sophisticated fraud detection (FD) technologies by CC companies. An effective and accurate FDS for real-time fraud identification can generally be divided into two main types: anomaly detection systems [11].

Financial entities issuing CCs or overseeing online payments must implement automatic FD mechanisms. This practice not only cuts down on financial losses but also boosts the confidence of their customers. Thanks to the advent of artificial intelligence and big data, there are now innovative opportunities to employ sophisticated ML algorithms for identifying fraudulent activities [14]. The current FD technologies leverage sophisticated DM, ML, and DL techniques to achieve high efficiency. These systems use a binary classification approach, utilizing datasets with labeled transactions—identifying them as normal or fraudulent. The model created through this process can determine new transactions' legitimacy. However, employing classification methods to identify fraudulent activities comes with its own set of challenges [15,16].

Automated mechanisms for identifying fraud play a crucial role for entities engaged in providing CC services or managing online payments, as they aid in reducing financial losses and enhancing trust among consumers. The advent of big data and advancements in artificial intelligence have paved the way for using complex ML models to spot fraudulent activities. Bao and colleagues (2022) [14] have demonstrated that the most recent advancements in fraud detection systems (FDSs), which utilize sophisticated DM, ML, and DL techniques, are highly efficient. The present study explores CCF, a critical type of banking fraud. CCs, a significant form of online payment globally, have made electronic transactions easier and resulted in increased fraud perpetrated by cyber crimes. The illegal utilization of CC systems or data, frequently without the card owner's awareness, is an escalating issue affecting many banks and financial institutions worldwide.

Feature selection (FS) stands as a crucial preprocessing step aimed at addressing the issue of irrelevant features detrimentally affecting the performance of ML models. This process involves pinpointing and removing unnecessary attributes to decrease the dimensionality of the feature sets without compromising the accuracy of the model's performance. Numerous strategies have been devised for classifying datasets, among which meta-heuristic optimization algorithms (MHOA) have stood out for their proficiency in solving a broad spectrum of optimization challenges [17].

MHOA stands for optimization techniques that aim to identify the best or nearly the best solutions to various optimization challenges. These methods are not dependent on derivatives, highlighting their ease of use, versatility, and ability to avoid getting stuck at local optimum points. MHOAs employ a stochastic approach, starting their optimization journey with solutions generated at random, in contrast to gradient search methods that rely on calculating derivatives within the search space. Their straightforwardness and simplicity, rooted in fundamental concepts and ease of implementation, render these

algorithms adaptable and straightforward to tailor to specific issues. A distinctive feature of MHOAs is their superior ability to avoid early convergence. Thanks to their stochastic nature, they can effectively operate as a dark box, evading local optima and extensively probing the search space [18–23].

Employing MHOAs to manage FS dilemmas can significantly mitigate obstacles during data analytics. FS plays a pivotal role in identifying pertinent features within imbalanced datasets, especially before the classification of CCF instances in large datasets. The key advantages of FS encompass easier data understanding, decreased time required for training, and addressing high dimensionality concerns. Bio-inspired algorithms, which excel in solving intricate and combinatorial challenges, have successfully identified CCF.

1.1. Motivations

This study addresses the challenge of data imbalance in CCFD by implementing advanced meta-heuristic optimization (MHO) techniques to refine classifier performance. Random forest (RF) and support vector machine (SVM) classifiers are utilized, leveraging their proven effectiveness across various domains [24–26]. The core of the experimentation involved 135 variations in 9 typical S-shaped and V-shaped transfer functions, each designed to enhance the performance of these MHO techniques.

These variants were assessed using CCF benchmark datasets from the Kaggle repository, with RF and SVM classifiers acting as fitness evaluators. This robust evaluation pinpointed the sailfish optimizer (SFO) as a standout among 15 renowned MHO algorithms, including brown-bear optimization (BBO), African vultures optimization (AVO), and others. SFO was particularly notable for its ability to reduce feature size by up to 90% while achieving classification accuracy as high as 97%.

The results of this investigation are critical for improving the security of CC transactions, thus enhancing e-commerce reliability worldwide. Utilizing the European credit cardholders dataset from Kaggle, the study demonstrated the algorithm's capability to identify critical features accurately and reliably. Each algorithm was tested through 30 separate executions using the RF and SVM classifiers, ensuring the consistency and reliability of the model in detecting CCF. This research provides a significant foundation for future enhancements in FD technologies.

1.2. Contributions

This study offers significant advancements in addressing the challenges of data imbalance in CCFD, as outlined in the following contributions:

1. Employing data from the European cardholder dataset on Kaggle, this work evaluates the performance of various techniques to address data imbalance. These techniques are assessed based on mean classification accuracy, the number of selected features, fitness values, and computational time.
2. Implementation and testing of 15 MHO algorithms, each enhanced by nine different transfer functions, are conducted to identify the most significant features for managing data imbalance within the dataset.
3. Evaluation of two machine learning techniques, random forest (RF) and support vector Machine (SVM), assessing their effectiveness on features selected by the MHO algorithms to validate robustness in improving model performance under conditions of data imbalance.
4. Document significant improvements in classification accuracy, notably with the sailfish optimizer combined with the random forest (SFO-RF) approach, achieving up to 97% accuracy. This highlights the effectiveness of the proposed methods in overcoming the challenges posed by imbalanced datasets.

1.3. Structure

The remainder of the paper is organized as follows: Section 2 introduces the current researches that handle FD. Section 3 presents the proposed techniques. Section 4 discusses

the findings and analysis of experimental results. Finally, Section 5 concludes the paper, outlining implications derived from the results.

2. Related Work

Many research studies have recently emerged, conducting reviews on existing strategies for FD and prevention, as noted in the scholarly works. This segment highlights various academic studies centered on FD, focusing on those addressing the issue within the framework of class imbalance challenges. A wide range of methods has been utilized to uncover fraudulent financial transactions. To effectively review the most relevant literature in this area, it is helpful to categorize the critical methodologies into several distinct groups, such as ML, strategies for detecting CCF, ensemble methods, feature ranking techniques, and methods for user authentication.

Zojaji et al. [27] have organized the methods for detecting fraud in CC transactions into two primary categories: supervised and unsupervised. They provided a comprehensive classification of the techniques mentioned in the studies, focusing on the categories and the applied datasets. However, they did not suggest any new methods themselves. On the other hand, Adewumi et al. [28] reviewed detecting fraud in online CC transactions with methods inspired by nature using ML. Their review spanned studies from 1997 to 2016, focusing primarily on the essential techniques and algorithms that emerged between 2010 and 2015 without delving into the methodologies. Similarly, Chilaka et al. [29] investigated methods for FD in CCs within the e-banking industry, focusing on pertinent studies from 2014 until 2019 to summarize the approaches taken. They concentrated on solutions that emphasized a quick, efficient response. However, their review was not conducted systematically and did not include a classification system.

Khalid et al. [30] has developed an ensemble approach using various ML algorithms to improve CCFD. This method leverages different algorithms' strengths to enhance the precision and dependability of FD mechanisms. Through detailed experiments and analysis, the authors illustrate how their method can effectively tackle the difficulties of identifying fraudulent activities in CC transactions.

Abdul et al. [31] introduced a federated learning (FL) framework tailored explicitly for detecting CCF, including methods for balancing data to enhance effectiveness. FL allows for training models on multiple decentralized data sources without the need to gather the data in a single location, thereby protecting data privacy. The researchers applied methodologies to overcome the challenge of uneven data distribution, a common issue in FD data. Their experiments and analysis show that their method not only maintains privacy but also significantly increases FD's accuracy by addressing the data imbalance.

Chen et al. [32] have proposed a method for CCFD that leverages sampling alongside self-supervised learning approaches. Intelligent sampling is used to refine the choice of samples for training, thereby enhancing the training process's efficiency. Meanwhile, self-supervised learning is applied to extract valuable features from data that have not been labeled, which is crucial for identifying illegal transactions. Through the research, the team has shown that their methodology improves the accuracy of FD, reduces the computational effort required, and lessens the dependency on labeled datasets.

Taha et al. [33] developed an intelligent technique for identifying fraud in CC transactions using a refined version of the light gradient boosting machine (LightGBM) algorithm. Known for its speed and precision in processing vast amounts of data, the authors further enhance LightGBM's effectiveness by fine-tuning its settings. Their research and testing reveal that this optimized method significantly improves the accuracy of FD while keeping the computational demands manageable. This strategy presents a viable option for banks and other financial entities aiming to upgrade their systems to detect fraudulent activities.

In their work, Rawashdeh et al. [34] developed an effective technique for identifying CCF, employing a combination of evolutionary algorithms for selecting the best features and random weight networks for classification. This innovative approach aims to enhance the FD precision with careful choice of the most significant features and optimize the network

weights by minimizing the effort consumed. The effectiveness of this method in accurately spotting CCF is demonstrated through various tests and assessments, highlighting the security in finance.

Kennedy et al. [35] outlined the issue of significant imbalance in the CCFD dataset by introducing a method for synthesizing class labels. This imbalance often results in models that are biased and ineffective at identifying fraudulent transactions. To address this, the team created synthetic examples of the less represented class, effectively evening out the dataset. They applied ML algorithms to these balanced datasets to train their models. Through experimental analysis, they showed that their method significantly enhances the performance of systems that detect CCF, making a noteworthy contribution to the FD domain.

Aziz et al. [36] delved into several DM approaches to identify CCF, concentrating on a range of ML tactics, including RF, SVM, hybrid methods, decision tree (DT), and DL. These techniques were employed to detect common patterns in consumer behavior from historical data. Their analysis of ML strategies revealed notable disparities across various studies and pointed out potential future directions for investigation.

Singh et al. [7] introduced a model for CCFD that integrates a two-stage process involving SVM and an optimization technique inspired by firefly behavior. Initially, the model employs the firefly technique alongside the CfsSubsetEval technique to refine the FS. Subsequently, an SVM classifier is utilized to build the CCFD model in the second stage. This approach achieved a classification with 85.65% accuracy in 591 transactions.

Nguyen et al. [37] utilized a DL strategy that incorporates long short-term memory (LSTM) and convolutional neural network techniques to successfully identify CCF on a variety of card types, including European, small, and tall, across various datasets. To combat the challenge of imbalanced classes, the research employed sampling methods, which, while reducing efficacy on unseen data, enhanced performance on familiar samples. The study demonstrates the proposed DL methods' capability to detect CCF in real-world applications, outperforming traditional ML models. Among all tested algorithms, the LSTM model with 50 units was highlighted for its superior performance, achieving an F1 score of 84.85%.

Ahmed et al. [38] explored using FS to identify intrusions in wireless sensor networks. They utilized particle swarm optimization (PSO) and principal component analysis space for this purpose, and they also compared its effectiveness with that of the genetic algorithm (GA). Rtayli et al. [39] developed an advanced method for identifying CC risk, employing algorithms such as DF and SVM to detect fraud.

Misra et al. [40] and Schlör et al. [41] have balanced datasets for detection models by applying under-sampling methods. A significant drawback of under-sampling is its propensity to exclude valuable instances from the training dataset, potentially diminishing detection accuracy. Conversely, isolation techniques have been utilized to approximate data distribution and construct a model with a diverse mixture of components. Such strategies for detecting outliers have proven effective in identifying fraud, as shown by Buschjäger et al. [42]. However, there is a notable absence in the research into comprehensive evaluations of the recent ML algorithms that leverage under-sampling to mitigate issues of imbalance in hybrid semi-supervised methods, which merge supervised learning with unsupervised outlier detection, which are significantly underutilized—additionally, the assessment of FDSs.

Hajek et al. (2022) [43] focused on creating FDSs utilizing XGBoost, which also evaluates the financial implications of such systems. This system underwent extensive testing on a dataset of over 6 million mobile transactions. To determine their model's effectiveness, they compared the proposed model to other ML strategies designed for managing imbalanced datasets and identifying outliers. Their research showed that a semi-supervised ensemble model, combining unsupervised outlier detection techniques with an XGBoost technique, surpassed the performance of other models regarding standard classification metrics. The most substantial cost reduction was achieved by integrating random under-sampling with the XGBoost approach.

Krim et al. [44] described an autoencoder as a particular kind of neural network that can encode and decode data similarly. This approach includes the specialized training of autoencoders on data points that are not anomalous. It depends on evaluating reconstruction errors to identify anomalies classified as either 'fraud' or 'no fraud.' This suggests that in situations not previously encountered by the system, there is a greater chance of detecting anomalies [2]. A slight rise beyond the maximum limit is typically marked as unusual. This method has been utilized in scenarios involving an autoencoder-based framework for identifying anomalies. Within the realm of ML, a generative adversarial network (GAN) consists of two neural networks collaborating to enhance their forecasting abilities. Mainly unsupervised, GANs develop by engaging in a collaborative zero-sum game approach.

These investigations utilized various ML techniques, such as SVM, DT, RF, NB, LR, LightGBM, and MLP. Moreover, firefly optimization, SMO, and hybrid sampling were employed. The findings are predominantly shown through accuracy (*ACC*), showcasing high levels of success in numerous cases. For example, Singh et al. reached an accuracy level of 85.65%, whereas Balogun et al. recorded accuracies of 97.50% with SVM and 98.60% with RF. Although various methods are used across these studies, a significant number have proven to be effective in detecting fraud.

The comprehensive analysis of the existing literature underlines a crucial realization: the landscape of FD, especially within the realm of CC transactions, is rapidly evolving in response to the equally dynamic tactics of fraudsters. The shift from traditional statistical methods to more intricate approaches, such as ML and MHO, marks a significant turning point in the battle against financial fraud. These advancements are not merely incremental; they are pivotal in addressing the persistent challenge of imbalanced data, which significantly undermines the effectiveness of detection systems. As such, the field stands on the brink of a new era in FD, where deploying sophisticated algorithms could redefine security standards in digital financial transactions. The ongoing innovation in this sector is vital, promising not only to enhance accuracy but also to fortify the resilience of economic systems against cyber-fraud threats.

3. Proposed Model

The diagram depicts a comprehensive workflow for FS in the context of FD using MHO algorithms. FS is a critical preprocessing step aimed at reducing the dimensionality of high-dimensional datasets by eliminating irrelevant and redundant features, thereby enhancing the performance of ML models. To address the class imbalance, we adopted the under-sampling technique. This involved randomly sampling an equal number of instances from the minority class (fraudulent transactions) and the majority class (non-fraudulent transactions). By equalizing the distribution of classes in the dataset, we aimed to enhance the performance of classification models in detecting fraudulent activities. The dataset was partitioned into subsets based on the class label, segregating fraud and non-fraud transactions. Then, an equivalent number of instances (492 instances) were randomly sampled from the majority class to match the minority class. The sampled subsets from both classes were combined to form a balanced dataset, which was shuffled to introduce randomness and prevent bias in subsequent analyses. The FS phase involves the application of different well-known MHO algorithms, including brown-bear optimization (BBO), African vultures optimization (AVO), Aquila optimization (AO), sparrow search algorithm (SSA), artificial bee colony (ABC), particle swarm optimization (PSO), bat algorithm (BA), grey wolf optimization (GWO), whale optimization algorithm (WOA), grasshopper optimization algorithm (GOA), sailfish optimizer (SFO), Harris hawks optimization (HHO), bird swarm algorithm (BSA), atom search optimization (ASO), and Henry gas solubility optimization (HGSO). These algorithms are designed to efficiently search for optimal feature subsets while considering the combinatorial nature of the problem and the exponential increase in computational time with problem complexity. The MHO techniques are evaluated against nine common S-shaped and V-shaped TFs to produce multiple variants. These variants are

assessed using random forest (RF) and support vector machine (SVM) classifiers as fitness evaluators. Finally, the performance of the top-performing algorithm variants for each classifier is compared with 15 MHO algorithms. This comparative analysis provides insights into the effectiveness and robustness of the MHO algorithms for FS in FD applications, as shown in Figure 2.

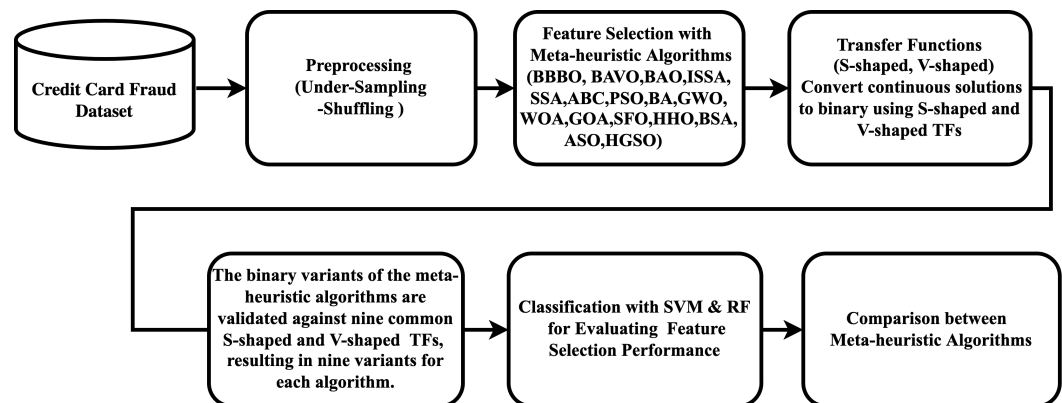


Figure 2. Framework of CCFD Model.

3.1. Data Preprocessing

The preprocessing steps involve addressing class imbalance and preparing the dataset for feature selection. An under-sampling technique is employed to handle the severe class imbalance in the dataset, where an equal number of instances from both fraudulent and non-fraudulent classes are randomly sampled, resulting in a balanced dataset. This ensures that the models are trained on representative data from both classes. Additionally, the dataset is partitioned based on the class label, and the sampled subsets are combined and shuffled to introduce randomness. Following the preprocessing steps, a balanced dataset was obtained, comprising 984 instances (492 frauds and 492 non-frauds), wherein the occurrences of both fraudulent and non-fraudulent transactions are approximately equal, as in Table 1. This balanced dataset is the foundation for subsequent analyses, which include feature engineering, model training, and evaluation.

Table 1. Original and balanced dataset.

Original DataSet	Attributes	Total Transactions	Fraud Instances	Non-Fraud Instances	Ratio
	30	284,807	492	284,315	0.172%
Balanced DataSet	Attributes	Total transactions	Fraud instances	non-Fraud instances	Ratio
	30	984	492	492	0.50%

3.2. MHO Algorithms for FS

The advantage of using MHO algorithms lies in their ability to pinpoint the key features within corpus data. In our recent research, we employed 15 distinct MHO algorithms to determine the crucial features needed for the accurate prediction of CCF and to identify which features, when eliminated, could improve or maintain the system's predictive performance at its peak. A brief description of each algorithm is provided below.

1. **Brown-Bear Optimization (BBO):** The BBO algorithm is an MHO approach to optimization, first presented in paper [18]. This method is unique because it is inspired by the specific abilities of brown bears to distinguish and sniff out scents, a trait not seen in other bear species. These special abilities have been translated into mathematical models to build the BBO technique, effectively replicating the natural behaviors of bears. Brown bears showcase significant intelligence through their ability to differentiate between various smells, using their sense of smell as a critical form of communication. They demonstrate pedal scent differentiation throughout their ter-

ritories, each group displaying unique behaviors. These include specific walking patterns, deliberate stepping, and manipulation of their feet on the ground, all contributing to their ability to distinguish scents. Moreover, brown bears exhibit a sniffing pedal differentiation behavior, where group members prefer to engage in sniffing activities. The efficiency of the optimization algorithm is dependent on its ability to exploit and explore. The BBO algorithm's exploitation aspect is inspired by the behavior of differentiating scents through pedals. In contrast, its exploration aspect is akin to the act of sniffing out differences in pedal scents.

2. **African Vultures Optimization (AVO):** AVO is a novel swarm-based optimization technique [45] inspired by hunting African vultures, who scavenge for weak animals and carcasses. These birds exhibit diverse traits and are classified into three groups based on strength, with the strongest having the highest chances of securing food. Vultures employ rotational flight to cover vast distances and locate food sources, often using aggressive tactics to access prey. The AVOA mimics these behaviors to optimize search processes in various problem-solving scenarios [46].
3. **Aquila Optimization (AO):** AO is introduced [47] as a novel MHO algorithm inspired by the hunting strategies and behaviors of the majestic Aquila genus, which includes eagles known for their keen vision, agility, and efficiency in capturing prey. The algorithm adapts these characteristics into a computational framework for solving the optimization. By mimicking the efficient hunting techniques of eagles, the Aquila optimizer aims to offer a powerful and practical approach to optimization tasks, potentially outperforming existing algorithms in terms of convergence speed, solution quality, and robustness across various problem domains.
4. **Sparrow Search Algorithm (SSA):** SSA [48] is an MHO method inspired by the social behavior and interactions of bird swarms, particularly sparrows. Sparrows, found globally and often living near human habitats, are omnivorous birds known for feeding on weed or grain seeds. They exhibit intelligence and memory, employing anti-predation and foraging behaviors. Captive sparrows are categorized into producers, actively seeking food sources, and scroungers, who obtain food from producers. Sparrows flexibly switch between these roles using similar foraging strategies. In SSA, each sparrow monitors the behavior of its neighbors, with attackers competing for high food intake for the flock. Sparrows utilize different foraging strategies to optimize energy utilization and increase food intake, with scrawny sparrows benefiting from these strategies. Sparrows in the search space are vulnerable to predator attacks and must seek safer locations. Sparrows exhibit natural curiosity and vigilance, emitting warning chirps to alert the group of danger, prompting them to fly away from potential threats. Based on the behaviors observed in sparrows, a mathematical model is formulated to construct the SSA algorithm, which leverages these principles to optimize search processes in various problem-solving scenarios [49].
5. **Artificial Bee Colony (ABC):** ABC, in 2005, was proposed by Karaboga's model [50] of the bee colony's foraging behavior. Inspired by the intelligent foraging behaviors of honey bees, the ABC algorithm's search process comprises three primary phases: dispatching forager bees to assess nectar quantity, sharing information with onlooker bees, and deploying scout bees to explore potential new food sources. This algorithm is part of a broader trend of algorithms inspired by insect colonies' foraging behavior under the "survival of the fittest" rule. This algorithm boasts easy implementation, minimal control parameters, and robust stability [51].
6. **Particle Swarm Optimization (PSO):** PSO is an effective and straightforward optimization technique inspired by the social behavior of animals like birds and fish. It has been widely applied across numerous fields, such as ML, image processing, data mining, robotics, etc. PSO was initially introduced by Eberhart and Kennedy in 1995 [52], drawing on models that mimic the collective behavior observed in natural species. As a result, PSO has found application across a broad range of industries for tackling various optimization challenges [53].

7. **Bat Algorithm (BA):** BA, a meta-heuristic approach inspired by the echolocation of bats, was introduced by Yang in 2010 [54]. This method draws inspiration from the echolocation behavior of microbats, which is characterized by varying pulse emission rates and loudness. Moreover, it incorporates principles of swarm intelligence (SI), influenced by observations of bats. Typically, bats utilize short, intense sound pulses during nocturnal hunts to locate obstacles or prey through the echoes these pulses generate. Additionally, the unique auditory system of bats enables them to ascertain the size and position of objects [55].
8. **Grey Wolf Optimization (GWO):** In 2014, Zorarpacı and Özel [56] introduced the GWO algorithm, which has since become one of the most used algorithms based on SI. The GWO algorithm's inspiration comes from grey wolves' natural hunting behavior, which efficiently tracks and captures their prey. The algorithm mimics the social hierarchy within a wolf pack to assign various roles during the optimization process. These roles are categorized into four groups: omega, alpha, delta, and beta, each signifying an optimal solution that has to be discovered [57].
9. **Whale Optimization Algorithm (WOA):** Mirjalili and Lewis developed WOA [58]. It was designed to tackle numerical optimization challenges. It incorporates three distinct mechanisms inspired by the feeding strategies of humpback whales: prey detection, prey encirclement, and bubble-net hunting. WOA aims to pinpoint the optimal solution for specific optimization issues by deploying a group of search agents. What sets WOA apart from similar algorithms are the unique rules it applies to enhance potential solutions at every step of optimization. Mimicking the predatory tactics of humpback whales, WOA zeroes in on and captures prey using a method referred to as bubble-net feeding [59].
10. **Grasshopper Optimization Algorithm (GOA):** Saremi et al. [60] introduced GOA, which draws inspiration from the natural foraging and swarming behavior of grasshoppers. This algorithm stands out due to its adaptive mechanism, balancing the exploration and exploitation processes. Due to these features, GOA has the potential to navigate the complexities of multi-objective search spaces more efficiently than other strategies. Moreover, it boasts a lower computational complexity than many current optimization methods [61].
11. **Sailfish Optimizer (SFO):** SFO [62] is a meta-heuristic algorithm that mimics the hunting behavior of sailfish preying on sardines. This hunting strategy aids predators in conserving energy. The algorithm features two populations: sailfish and sardines. The sailfish represent candidate solutions, with their positions in the search space corresponding to problem variables. SFO aims to randomize the movement of both the sailfish and sardines. Sailfish are dispersed throughout the search area, while the positioning of sardines assists in locating the best solution within the search space [63].
12. **Harris Hawks Optimization (HHO):** HHO is a developed population-based optimization method inspired by the cooperative hunting behavior of Harris hawks in nature. Heidari et al. [64] introduced HHO to simulate the dynamic teamwork and hunting strategies of these hawks, which include techniques such as tracing, encircling, approaching, and attacking prey. In this model, the hawks' pursuit efforts represent agents navigating the search area, with the prey representing the optimal solution. HHO effectively addresses various real-world optimization challenges and can handle discrete and continuous domains. It can explore uncharted search spaces and achieve high-quality solutions, making it suitable for tasks requiring optimal parameter extraction. Overall, HHO demonstrates promising performance and offers a novel approach to solving optimization problems inspired by nature's cooperative behaviors [65].
13. **Bird Swarm Algorithm (BSA):** Meng et al. [66] unveiled a novel MHO strategy known as BSA for tackling continuous optimization issues. This approach is inspired by SI, which originates from the collective behaviors and interactions observed in bird swarms. By emulating the search for food, vigilance, and flight patterns of

birds, BSA effectively leverages SI drawn from these avian swarms to address various optimization challenges [67].

14. **Atom Search Optimization (ASO):** ASO is presented as an optimization method inspired by molecular dynamics. In this approach, the search space is navigated by the position of atoms, each representing a potential solution, evaluated based on its mass or “heaviness” [68]. The interaction between atoms is determined by their proximity, leading to either attraction or repulsion. This dynamic causes lighter atoms to move towards the heavier ones. Additionally, heavier atoms have a slower movement, so they are more efficient in thoroughly searching local areas for improved solutions. On the other hand, the rapid movement of lighter atoms enables them to explore new and broader areas of the search space more effectively.
15. **Henry Gas Solubility Optimization (HGSO):** Hashim et al. introduced [69] the HGSO algorithm in 2019. HGSO is a variant of the MHO algorithm, drawing inspiration from Henry’s law to mimic the behavior of gas particles [70]. It employs gas clustering behavior to effectively balance exploitation and exploration within the search space, thus mitigating the risk of converging to local optima [71].

3.3. ML Techniques

This section outlines the ML classifiers employed in the research to evaluate the subset of selected features regarding classification accuracy and fitness values.

1. **Random Forest (RF)** In 2001, Breiman [72] introduced the RF algorithm, incorporating the outcomes of multiple decision trees during training to derive the mode of the classes or the average prediction of the individual trees. It deliberately selects samples and predictors randomly, and the optimized hyperparameter values have a substantial impact on enhancing prediction accuracy [73]. The fundamental concept of the RF algorithm revolves around the bootstrap sampling method, which randomly and repeatedly generates N samples from the training dataset. It builds a robust classifier by integrating weak decision trees. Two thirds of the original data size will be used for each training set cluster and one third for test sets. The features were selected randomly to split decision trees, and RF helps to relieve overfitting problems by choosing input and predictors randomly [74]. The strengths of RF are that the learning effect for integrated RF is regularly more significant than the sum of the learning effect for parts. RF is highly versatile, excelling with large volumes of data, effectively estimating missing values, and balancing errors in unbalanced sets. It accurately identifies the most critical features in classification and can be utilized with other datasets for classification and regression tasks. Furthermore, it enhances prediction accuracy without significantly increasing computation and demonstrates strong performance in predicting stock prices. Its wide-ranging applications include bioinformatics, data mining, big data, and various other domains [75].
2. **Support Vector Machine (SVM)**
Vapnik and Cortes [76] first proposed SVM for classifying linear and non-linear problems. It is a well-known supervised binary classifier that builds a model by grouping data into two classes. It plots a hyperplane boundary to separate the training dataset (a decision surface) in the input space by maximizing the isolation edge between positive and negative examples [77]. The extensive research has thoroughly explored SVM’s ability to optimize performance by adapting the training classifier or reducing the training set size. SVM holds crucial applications in various fields, including ML, statistics, object recognition, text categorization, speaker identification, and health care [78].

3.4. Transfer Functions (TFs)

As the final solution acquired through different utilized MHO techniques comprises continuous values, MHO techniques cannot address an FS problem directly. Thus, it becomes essential to employ a mapping (transfer) function to convert these continuous

values into binary 0 s or 1 s. Transfer functions (TFs) [79] dictate the rate of change in the decision variable values from 0 to 1 and vice versa. Two shared transfer functions used for this purpose are S-shaped, which are so named because their graphical representation resembles the letter 'S', and V-shaped transfer functions. S-shaped functions are used to map continuous values to probabilities, which can then be converted into binary values through a thresholding process. The output of an S-shaped transfer function lies between 0 and 1, representing the probability of including a feature. V-shaped transfer functions, on the other hand, are characterized by their 'V'-shaped graphical representation. These functions are also used to map continuous values to binary decisions but follow a mathematical approach different from that of S-shaped functions. The output of a V-shaped transfer function is used to determine the likelihood of a feature's inclusion based on a different set of criteria.

When selecting a TF for the conversion of continuous to binary values, several considerations must be taken into account from the MHO techniques perspective, as follows:

- The range of values from a TF should be between 0 and 1, representing the probability of a feature-changing state.
- If the evaluation metric for the feature indicates suboptimal performance, the TF should show a higher probability of changing the current state in the next iteration.
- When a feature is considered optimal, the TF should have a low probability of changing its current state.
- The probability generated by the TF should rise as the evaluation metric approaches a threshold value. This enables less optimal features to have a higher likelihood of changing their state, which helps move towards more optimal solutions in subsequent iterations.
- The probability derived from a TF must decrease as the evaluation metric moves away from the threshold value.

These concepts demonstrate the high capability of TFs to convert the process of continuous search into binary for each $\mathbf{x}$, using Equation (1):

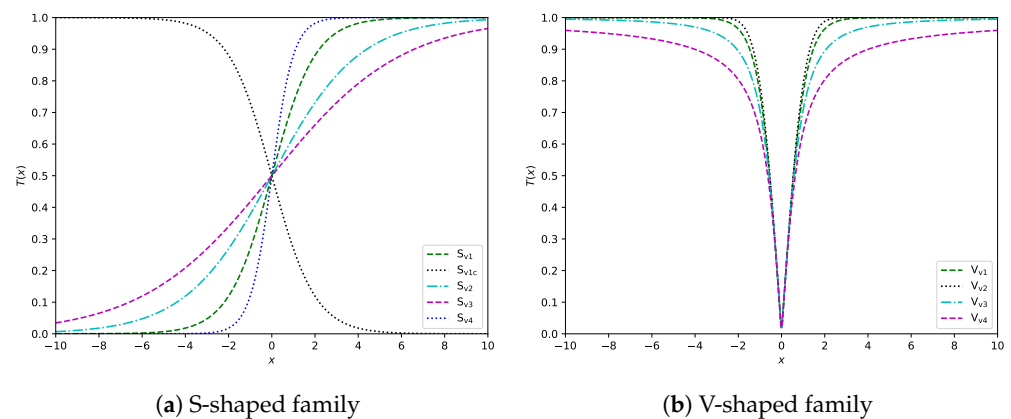
$$(x_{i,j}^{t+1})_{\text{bin}} = \begin{cases} \begin{cases} 0 & \text{if } rand < TF(x_{i,j}^{t+1}), \\ 1 & \text{if } rand \geq TF(x_{i,j}^{t+1}), \end{cases} & \text{if TF is S-shaped,} \\ \begin{cases} \neg(x_{i,j}^t)_{\text{bin}} & \text{if } rand < TF(x_{i,j}^{t+1}), \\ (x_{i,j}^t)_{\text{bin}} & \text{if } rand \geq TF(x_{i,j}^{t+1}), \end{cases} & \text{if TF is V-shaped,} \end{cases} \quad (1)$$

where $(x_{i,j}^{t+1})_{\text{bin}}$ represents the j -th dimension of the i -th individual at the current iteration $t + 1$, $rand$ is a number selected randomly from within the range $[0, 1]$, and $TF(x_{i,j}^{t+1})$ is the probability value obtained when applying a given TF to every j -th component's continuous value of agent i . It is clear from Equation (1) that we have two cases: (i) if the TF is S-shaped, then if $rand$ is less than the probability returned by the involved TF, the j -th dimension of the original individual is set to 0; otherwise, it is set to 1; and (ii) if the TF is V-shaped, then if $rand$ is less than the probability returned by the involved TF, the j -th dimension is negated; otherwise, it remains unchanged. Thus, continuous variables are successfully mapped into binary by using the S-shaped and V-shaped TFs and Equation (1).

Table 2 reports the families of TFs, while Figure 3 exhibits these two families, divided into S-shaped and V-shaped TFs. Here, it should be noted that the proposed MHO techniques were evaluated based on the nine TFs whose mathematical expressions are shown in the Table 2.

Table 2. S-shaped and V-shaped TFs families.

S-Shaped Family		V-Shaped Family	
Name	Transfer Function	Name	Transfer Function
S_{v1}	$TF(x) = \frac{1}{1+exp^{-x}}$ [80]	V_{v1}	$TF(x) = \tanh(x) $ [81]
S_{v1c}	$TF(x) = \frac{1}{1+exp^x}$	V_{v2}	$TF(x) = \left \operatorname{erf}\left(\frac{\sqrt{\pi}}{2}x\right) \right $
S_{v2}	$TF(x) = \frac{1}{1+exp^{(-x/2)}}$	V_{v3}	$TF(x) = \left (x)/\sqrt{1+x^2} \right $
S_{v3}	$TF(x) = \frac{1}{1+exp^{(-x/3)}}$	V_{v4}	$TF(x) = \left \frac{2}{\pi} \arctan\left(\frac{\pi}{2}x\right) \right $
S_{v4}	$TF(x) = \frac{1}{1+exp^{-2x}}$		

**Figure 3.** Families of transfer functions.

3.5. Sampling Technique

The dataset used in this study contains 284,807 transactions with a severe class imbalance, where only 492 instances are fraudulent. Under-sampling was chosen because it creates a balanced dataset by reducing the majority of class instances without introducing synthetic data points. This methodological approach aligns to accurately capture the inherent characteristics of both fraudulent and non-fraudulent transactions. Given the computational efficiency and simplicity of implementation, under-sampling is suitable for handling large-scale datasets like ours. It allows us to focus on the intrinsic patterns within the data without the added complexity of generating synthetic instances, which may not accurately represent real-world fraud scenarios. Under-sampling is one of several techniques data scientists can use to extract more accurate information from originally unbalanced datasets [82]. **The steps taken for the under-sampling process are as follows:**

- **Random Sampling:** Instances from the majority class (non-fraudulent transactions) were randomly sampled to match the instances in the minority class (fraudulent transactions), resulting in a balanced dataset.
- **Data Partitioning:** The dataset was partitioned based on the class label, ensuring equal representation from both classes.
- **Combining and Shuffling:** The sampled subsets were then combined and shuffled to introduce randomness and prevent ordering bias.

While reducing the dataset size to 492 instances from the original 284,807 transactions does involve a reduction in the overall data volume, our approach ensured that this reduction was performed in a manner that preserved the integrity and representativeness of the data:

- We employed stratified sampling to ensure that the under-sampling process maintained a proportional representation of both fraud and non-fraud within the reduced dataset. This approach helps mitigate the risk of losing critical information that may be present in the minority class.

- Post-sampling, rigorous feature engineering, and feature selection techniques (MHO) maximize the retained information's relevance and quality. Focusing on the most discriminative features, we aimed to capture and leverage the essential characteristics that differentiate fraudulent from non-fraudulent transactions.

4. Experimental Methodology

This section presents the performance evaluation results of 15 MHO techniques and their counterparts with different ML classifiers RF and SVM based on nine TFs. The experimental dataset was downloaded from the CCF online dataset on the Kaggle-ML Repository. The parameters of the utilized MHO methods and ML classifiers are defined in Section 4.2. Section 4.3 states the utilized Performance Metrics. The experimental results are discussed in Sections 4.4 and 4.5. The convergence curves are shown in Section 4.7.

4.1. Dataset Description

To assess the robustness of the MHO techniques with nine different TFs (five S_{Shaped} and four V_{Shaped} functions), the Kaggle dataset was considered in this research. It can be downloaded directly from the Kaggle repository (<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>) (accessed on 16 July 2024). This dataset was used to create a framework for CCFD. More relevant autonomous information functions and target yield markers are extracted and utilized to detect fraud. The dataset contains transactions made by credit-cardholders in September 2013, primarily from European cardholders. It includes 284,807 transactions over two days, of which only 492 transactions are recorded as fraud. This creates a severe class imbalance, with fraudulent transactions accounting for only 0.172% of all transactions. The dataset consists of numerical input variables resulting from a principal component analysis (PCA) transformation. Due to confidentiality constraints, the original features and additional background information about the data are unavailable. The features V1 through V28 represent the principal components obtained through PCA, while 'Time' and 'Amount' are the only features not subjected to PCA transformation. 'Time' means the elapsed time in seconds between each transaction and the first transaction in the dataset, while 'Amount' denotes the monetary value of each transaction. The objective class is divided into two categories: 1 for fraudulent transactions and 0 for non-fraudulent transactions Table 3. We explored various ML techniques using binary or multiclass datasets in the Python programming language.

Table 3. CCF Dataset Description.

No.	Feature	Description
1	Time	Time (in seconds) required to calculate the intervals between the current and first transaction
2	V1, V2, ... V28	These attributes show the result of PCA dimensionality reduction to protect user identities and sensitive features
3	Amount	Amount of transaction
4	Class label	Binary class label 0 or 1 for fraudulent and non-fraudulent

4.2. Parameter Settings

Several MHO algorithms based on two ML classifiers were evaluated utilizing 9 TFs. These MHO algorithms include BBO, AVO, AO, SSA, ABC, PSO, BA, GWO, WOA, GOA, SFO, HHO, BSA, ASO, and HGSO. Each technique underwent thirty experiments on the utilized dataset due to the stochastic character of the Meta-heuristic techniques. We documented evaluation metrics based on average results to make a fair comparison between different methods. We assigned a population size of 10 and a maximum of 100 iterations to all techniques. The benchmark reflects the problem size by the number of features, and the search domain is set to $[-1, 1]$, allowing exploration within a constrained space.

Our framework used a ten-fold cross-validation method to ensure the outcomes' reliability. This approach involves randomly dividing the benchmark into training and testing subsets. The training subset comprises 80% of the data and is used to learn the

machine learning model, and the test dataset evaluates the selected attributes. Each method's configurations and parameter standards were based on their initial versions and information from their primary publications, as introduced in Table 4. Our computing environment utilized Python 3.10, an Intel Core i7 processor, 16 GB of RAM, and an NVIDIA GTX 1050i GPU.

Table 4. Parameter configurations of all methods.

Methods	Parameters
All methods	The number of runs = 30 The number of allowed Iterations $G_{max} = 100$ The size of population $M = 10$ Dimensionality D = The attributes number in the utilized dataset
BBO [18]	X^{LB} is the smaller bounds of parameters X^{LB} is the bigger bounds of parameters
AVO [45]	$L_1 = 0.7$ $L_2 = 0.2$ $w = 2$ $P_1 = 0.6$ $P_2 = 0.6$ $P_3 = 0.5$
AO	Scale factor whose value is defined by the scales of the problem $s = 0.01$ $\beta = 1.5$ Number of search cycles $r1 = 10$ $U = 0.00565$ $\omega = 0.005$ Adjustment parameters for exploitation stage $\alpha = 0.1$ and $\delta = 0.1$ Aquila's random movements $G_1 \in [-1, 1]$ Flying slope of Aquila $G_2 \in [2, 0]$
SSA	Scroungers' number $SD = 0.1 * N$ Producers; number $PD = 0.2 * N$ The iterations' number in LSA = 20 Safety threshold $ST = 0.8$
ABC	Number of employed bees = 16 Number of scout bees = 3 Number of onlooker bees = 4
PSO	Inertia weight ($\omega_{max} = 0.9\omega_{min} = 0.4$) Acceleration coefficients ($c_2 = c_1 = 1.2$)
BA	Loudness $A = 0.8$ Lower and upper pulse frequencies = 0, 10 Pulse emission rate $r = 0.95$
GWO	a is linearly reduced from 2 to 0
WOA	a is linearly reduced from 2 to 0 $b = 1.0$ $p = 0.5$
GOA	$C_{min} = 0.00004$ and $C_{max} = 1$
SFO	Ratio between sardines and sailfish $pp = 0.1$ $\varepsilon = 0.0001$ $A = 1$
HHO	Rabbit energy $E \in [-1, 1]$
BSA	Frequency of flight $ff = 10$ Followed coefficient $fl = 0.5$ Effect on birds' vigilance behaviors ($a_1 = a_2 = 1.0$) Acceleration coefficients ($c_1 = c_2 = 1.5$) Probability of foraging for food $p = 0.8$
ASO	Depth weight $\alpha = 50$ Multiplier weight $\beta = 0.2$
HGSO	Number of clusters = 2 $l_1 = 5E - 03$, $l_2 = 1E + 02$, and $l_3 = 1E - 02$ $\alpha = \beta = 0.1$ and $K = 1.0$

4.3. Performance Metrics

The benchmark data are determined 30 times to compare the effectiveness of 15 MHO algorithms with RF and SVM classifiers. The following are the FS methodology evaluation metrics that were utilized.

- **The average accuracy ((AVG_{ACC}))** is evaluated by executing the method for 30 runs and calculating the percentage of correct data classification. The accuracy is determined using the following equation:

$$AVG_{ACC} = \frac{1}{30} \frac{1}{m} \sum_{k=1}^{30} \sum_{r=1}^m match(PL_r, AL_r), \quad (2)$$

where m represents the number of samples in the test subset, PL_r and AL_r , respectively, indicate the predicted and reference class labels for sample r . The comparison function $match(PL_r, AL_r)$ determines the matching between the predicted and the reference label. If they match, then $match(PL_r, AL_r)$ equals 1; otherwise, it equals 0.

- **The average fitness outcomes (AVG_{Fit})** are evaluated by implementing the approach in 30 individual trials and unequivocally demonstrate that decreasing the selected attributes' number and increasing the accuracy rate synergistically work together. It is crucial to note that the ideal outcome is determined by the following formula: the lowest value represents the best result.

$$AVG_{Fit} = \frac{1}{30} \sum_{k=1}^{30} f_*^k, \quad (3)$$

where f_*^k is the optimal fitness value obtained in the k -th run.

- **The average number of features chosen ($AVG_{Features}$)** indicates the average number of chosen features by implementing the methodology individually 30 times and is represented as

$$AVG_{Features} = \frac{1}{30} \sum_{k=1}^{30} \frac{|d_*^k|}{|D|}, \quad (4)$$

where $|d_*^k|$ is the length of features chosen in the optimal solution for the k -th run, and $|D|$ is the entire length of attributes in the utilized dataset.

- **The average computational time (T_z)** shows the execution time in seconds for each algorithm validated over 30 different runs and is represented as (5)

$$T = \frac{1}{N} \sum_{i=1}^N RunTime_i, \quad (5)$$

where N is the runs' number, and $RunTime_i$ is the computational time in seconds at run i .

- **Standard Deviation (STDE):** The average results from the thirty runs of the algorithm on the used dataset are assessed for stability as

$$STDE = \sqrt{\frac{1}{29} \sum_{k=1}^{30} (Y_*^k - Mean_Y)^2}, \quad (6)$$

where Y is the metric to be assessed, Y_*^k is the metric value Y in the k -th run, and $Mean_Y$ is the average of the metric over 30 independent runs.

In the following sections, we examine the analytical results, highlighting the most promising outcomes in bold.

4.4. Comparisons Based on RF Classifier Using CCF Dataset

In this section, we compare the performance of the fifteen MHO techniques based on the RF classifier. This evaluates the average classification accuracy, the number of selected features, and the average computational time. The aim is to assess the impact of the MHO algorithms in choosing the most relevant features.

Firstly, the classification accuracy is estimated using the original RF classifier without utilizing MHO techniques based on the number of features in the CCF dataset. The original RF classifier achieved 0.9328 classification accuracy. On the other hand, Table 5 shows the performance analysis of the RF classifier with 15 MHO techniques regarding different TFs based on the average classification accuracy to evaluate the impact of the MHO algorithms on the utilized dataset. Remarkably, SFO-RF ranked first with all TFs except S_V4 (S_V1 achieved 0.9762, S_V1C achieved 0.9754, S_V2 achieved 0.9773, S_V3 achieved 0.9770, V_V1 achieved 0.9779, V_V2 achieved 0.9768, V_V3 achieved 0.9756, and V_V4 achieved 0.9765). BBO-RF ranked first regarding S_V4 by obtaining 0.9754 classification accuracy.

Secondly, Table 6 shows the performance analysis of the RF classifier with 15 MHO techniques regarding different TFs based on the average number of selected features on the utilized dataset. It is remarkable that ($AVG_{Features}$) based on the PSO-RF according to S_V1 selected 5.0667 features, WOA-RF regarding S_V1C selected 9.8667 features, SFO-RF regarding S_V2 selected 6.8667 features; furthermore, SFO-RF regarding S_V3 selected 7.1333 features, PSO-RF regarding S_V4 selected 4.6000 features, AO-RF regarding V_V1 selected 11.3667 features, HHO-RF regarding V_V2 selected 11.8000 features, AO-RF and PSO-SSA regarding V_V3 selected 11.8333 features, and AVO-RF regarding V_V4 selected 11.5667 features. Therefore, PSO-RF regarding S_V1 selected the least number of chosen features.

Table 5. Assessment of the impact of 9 TFs on 15 MHO algorithms with RF in terms of (AVG_{ACC}).

Metric	BBO	AVO	AO	SSA	ABC	PSO	BA	GWO	WOA	GOA	SFO	HHO	BSA	ASO	HGSO	Winner
<i>S_{Shaped}</i>																
S_V1 AVG	0.9742	0.9700	0.9689	0.9692	0.9756	0.9675	0.9658	0.9717	0.9706	0.9703	0.9762	0.9689	0.9700	0.9700	0.9625	SFO
STDE	0.0037	0.0047	0.0044	0.0045	0.0025	0.0047	0.0061	0.0046	0.0042	0.0044	0.0044	0.0044	0.0047	0.0060	0.0042	
S_V1C AVG	0.9734	0.9717	0.9681	0.9695	0.9742	0.9689	0.9655	0.9717	0.9697	0.9720	0.9754	0.9703	0.9714	0.9689	0.9641	SFO
STDE	0.0044	0.0051	0.0040	0.0046	0.0037	0.0044	0.0055	0.0055	0.0051	0.0050	0.0030	0.0052	0.0047	0.0049	0.0053	
S_V2 AVG	0.9745	0.9709	0.9703	0.9706	0.9756	0.9692	0.9655	0.9711	0.9703	0.9700	0.9773	0.9689	0.9711	0.9725	0.9636	SFO
STDE	0.0034	0.0047	0.0052	0.0042	0.0045	0.0040	0.0063	0.0047	0.0047	0.0056	0.0039	0.0039	0.0052	0.0065	0.0050	
S_V3 AVG	0.9745	0.9706	0.9683	0.9706	0.9748	0.9714	0.9664	0.9709	0.9689	0.9723	0.9770	0.9692	0.9717	0.9717	0.9655	SFO
STDE	0.0046	0.0042	0.0036	0.0047	0.0038	0.0047	0.0053	0.0047	0.0039	0.0058	0.0037	0.0040	0.0040	0.0055	0.0059	
S_V4 AVG	0.9754	0.9706	0.9700	0.9706	0.9748	0.9644	0.9616	0.9714	0.9695	0.9714	0.9742	0.9686	0.9723	0.9661	0.9625	BBO
STDE	0.0037	0.0056	0.0052	0.0047	0.0031	0.0047	0.0077	0.0041	0.0055	0.0060	0.0048	0.0037	0.0049	0.0046	0.0052	
<i>V_{Shaped}</i>																
V_V1 AVG	0.9768	0.9720	0.9703	0.9711	0.9745	0.9675	0.9608	0.9734	0.9697	0.9709	0.9779	0.9700	0.9717	0.9594	0.9630	SFO
STDE	0.0036	0.0050	0.0060	0.0042	0.0046	0.0052	0.0066	0.0049	0.0060	0.0047	0.0040	0.0056	0.0051	0.0044	0.0051	
V_V2 AVG	0.9756	0.9725	0.9711	0.9723	0.9751	0.9672	0.9611	0.9737	0.9697	0.9703	0.9768	0.9695	0.9695	0.9613	0.9641	SFO
STDE	0.0025	0.0043	0.0064	0.0044	0.0051	0.0055	0.0098	0.0052	0.0051	0.0052	0.0036	0.0055	0.0046	0.0067	0.0057	
V_V3 AVG	0.9742	0.9725	0.9711	0.9731	0.9751	0.9692	0.9619	0.9734	0.9711	0.9734	0.9756	0.9686	0.9717	0.9641	0.9619	SFO
STDE	0.0030	0.0053	0.0047	0.0046	0.0046	0.0066	0.0064	0.0054	0.0047	0.0054	0.0025	0.0043	0.0046	0.0068	0.0047	
V_V4 AVG	0.9765	0.9734	0.9717	0.9714	0.9745	0.9683	0.9636	0.9709	0.9703	0.9731	0.9765	0.9711	0.9717	0.9641	0.9622	SFO
STDE	0.0050	0.0049	0.0046	0.0047	0.0040	0.0052	0.0063	0.0052	0.0047	0.0055	0.0034	0.0064	0.0046	0.0053	0.0052	

Bold values indicate the highest results obtained.

Table 6. Assessment of the impact of 9 TFs on 15 MHO algorithms with RF in terms of ($AVG_{Features}$).

Metric	BBO	AVO	AO	SSA	ABC	PSO	BA	GWO	WOA	GOA	SFO	HHO	BSA	ASO	HGSO	Winner
<i>S_{Shaped}</i>																
S_V1 AVG	12.5333	10.2000	11.2667	9.1667	12.5333	5.0667	10.8000	10.8667	10.9000	13.5667	7.7333	12.3333	12.8667	12.9000	12.5000	PSO
STDE	2.3055	3.9362	3.9067	4.1879	2.0122	1.8607	3.8419	3.2530	5.8728	2.5519	3.9911	2.6499	2.6297	1.4457	1.7464	
S_V1C AVG	12.4000	11.0000	12.1000	10.3667	12.6667	20.4333	13.8333	11.0667	9.8667	12.5667	13.3000	12.7000	13.0000	12.7667	13.2667	WOA
STDE	2.4440	4.3359	2.8325	3.7371	2.1344	3.6119	3.3673	2.6449	5.6079	1.7641	4.3217	2.4242	1.6330	2.6793	2.0645	
S_V2 AVG	12.7667	10.4667	10.9333	10.8333	12.2667	7.9333	12.8667	12.0000	10.5000	13.2667	6.8667	13.2000	12.0667	12.2333	14.0000	SFO
STDE	2.7891	3.5471	4.0982	3.8217	1.7114	2.6700	3.4712	2.6331	4.6673	2.1281	2.1868	2.9371	2.3371	1.9947	2.2211	
S_V3 AVG	12.4667	10.6000	10.4667	11.1333	12.8333	9.0000	12.7667	12.4333	11.6000	13.8667	7.1333	13.1667	12.8667	12.5000	13.5667	SFO
STDE	2.5914	3.6019	3.5094	3.7571	2.4233	2.4083	2.6543	3.0186	3.8523	2.7293	2.9067	2.4506	2.5785	1.9451	2.4723	
S_V4 AVG	12.3333	8.2333	11.7667	10.5667	12.5000	4.6000	11.6333	10.2667	9.6333	13.5000	7.4667	11.9667	12.5667	12.7667	13.2000	PSO
STDE	2.0385	3.4028	3.3733	5.2895	1.9958	1.8184	4.2543	3.3160	5.5226	2.2620	2.9970	2.9494	2.1084	2.6036	2.6758	
<i>V_{Shaped}</i>																
V_V1 AVG	12.1333	12.7333	11.3667	12.6667	14.7333	13.6333	13.3667	12.7667	12.5000	13.1333	12.5667	12.4333	13.1667	13.6667	14.7333	AO
STDE	2.3907	2.6196	2.0410	2.1029	2.6575	2.7016	2.7016	2.6164	2.8607	2.3485	2.6793	2.8129	2.3393	2.4129	2.7921	
V_V2 AVG	12.8000	12.7333	12.0000	13.2333	13.9667	13.5000	13.5333	12.4667	13.3000	12.7333	13.0000	11.8000	12.7000	13.8667	15.5000	HHO
STDE	2.6255	2.4891	2.3805	2.2013	2.1523	2.2023	2.8721	2.3627	2.4379	2.7681	2.2361	2.5351	2.3826	2.0613	2.5265	
V_V3 AVG	12.1333	12.1000	11.8333	11.8333	13.2667	14.1333	14.1000	13.1000	12.1667	13.3000	11.8667	12.3333	13.3667	13.9667	14.8667	AO, SSA
STDE	2.1092	2.1656	2.9107	2.5309	1.6519	2.6675	2.3431	2.7610	2.5831	2.6476	2.2020	2.7121	3.2092	2.4424	2.5263	
V_V4 AVG	12.0333	11.5667	12.9000	12.2333	13.5333	12.9333	13.7333	12.2000	12.4667	12.7333	12.1667	12.5000	12.4667	13.3333	16.0667	AVO
STDE	2.3307	2.0114	3.0914	2.2164	2.8952	2.4622	2.8394	2.4000	2.7293	2.2794	2.8057	2.0936	2.3200	2.5863	2.6949	

Bold values indicate the highest results obtained.

Thirdly, Table 7 shows the performance analysis of the RF classifier with 15 MHO techniques in terms of different TFs based on the average classification fitness to evaluate the impact of the MHO algorithms on the utilized dataset. Remarkably, AVG_{Fit} based on the SFO-RF method ranked first with all TFs except V_V4 (S_V1 achieved 0.0262, S_V1C achieved 0.0290, S_V2 achieved 0.0248, S_V3 achieved 0.0252, S_V4 achieved 0.0281, V_V1 achieved 0.0262, V_V2 achieved 0.0275, and V_V3 achieved 0.0282). BBO-RF ranked first regarding V_V4 .

Table 7. Assessment of the impact of 9 TFs on 15 MHO algorithms with RF in terms of (AVG_{Fit}).

Metric	BBO	AVO	AO	SSA	ABC	PSO	BA	GWO	WOA	GOA	SFO	HHO	BSA	ASO	HGSO	Winner
<i>S_{Shaped}</i>																
S_V1 AVG	0.0298	0.0332	0.0347	0.0337	0.0284	0.0339	0.0376	0.0318	0.0329	0.0341	0.0262	0.0350	0.0341	0.0341	0.0415	SFO
STDE	0.0036	0.0047	0.0044	0.0044	0.0025	0.0044	0.0059	0.0043	0.0047	0.0038	0.0041	0.0040	0.0043	0.0059	0.0041	
S_V1C AVG	0.0306	0.0318	0.0358	0.0338	0.0299	0.0378	0.0389	0.0318	0.0334	0.0321	0.0290	0.0338	0.0328	0.0352	0.0401	SFO
STDE	0.0041	0.0052	0.0038	0.0047	0.0035	0.0044	0.0054	0.0055	0.0054	0.0048	0.0032	0.0050	0.0043	0.0049	0.0051	
S_V2 AVG	0.0296	0.0324	0.0332	0.0329	0.0284	0.0332	0.0385	0.0327	0.0330	0.0342	0.0248	0.0353	0.0327	0.0314	0.0409	SFO
STDE	0.0031	0.0046	0.0045	0.0040	0.0044	0.0035	0.0060	0.0047	0.0048	0.0051	0.0038	0.0039	0.0050	0.0062	0.0050	
S_V3 AVG	0.0295	0.0328	0.0349	0.0330	0.0294	0.0314	0.0377	0.0331	0.0348	0.0322	0.0252	0.0350	0.0324	0.0323	0.0388	SFO
STDE	0.0041	0.0042	0.0033	0.0042	0.0037	0.0045	0.0053	0.0041	0.0034	0.0054	0.0036	0.0039	0.0038	0.0053	0.0057	
S_V4 AVG	0.0287	0.0320	0.0337	0.0328	0.0293	0.0368	0.0420	0.0318	0.0335	0.0329	0.0261	0.0352	0.0318	0.0380	0.0417	SFO
STDE	0.0037	0.0056	0.0050	0.0048	0.0029	0.0045	0.0081	0.0040	0.0058	0.0057	0.0041	0.0033	0.0048	0.0045	0.0051	
<i>V_{Shaped}</i>																
V_V1 AVG	0.0272	0.0321	0.0333	0.0329	0.0303	0.0369	0.0434	0.0307	0.0343	0.0334	0.0262	0.0340	0.0325	0.0449	0.0417	SFO
STDE	0.0033	0.0051	0.0057	0.0040	0.0043	0.0051	0.0066	0.0045	0.0057	0.0041	0.0035	0.0052	0.0048	0.0042	0.0048	
V_V2 AVG	0.0285	0.0316	0.0327	0.0320	0.0295	0.0371	0.0432	0.0304	0.0345	0.0338	0.0275	0.0343	0.0346	0.0431	0.0408	SFO
STDE	0.0026	0.0039	0.0062	0.0042	0.0048	0.0051	0.0094	0.0048	0.0049	0.0049	0.0035	0.0055	0.0045	0.0065	0.0054	
V_V3 AVG	0.0297	0.0313	0.0326	0.0307	0.0293	0.0354	0.0426	0.0309	0.0328	0.0309	0.0282	0.0353	0.0326	0.0403	0.0428	SFO
STDE	0.0029	0.0053	0.0046	0.0044	0.0045	0.0064	0.0063	0.0054	0.0046	0.0051	0.0023	0.0043	0.0042	0.0065	0.0043	
V_V4 AVG	0.0274	0.0303	0.0325	0.0325	0.0299	0.0358	0.0408	0.0330	0.0337	0.0310	0.0275	0.0329	0.0323	0.0401	0.0430	BBO
STDE	0.0046	0.0048	0.0044	0.0045	0.0039	0.0049	0.0061	0.0049	0.0043	0.0054	0.0028	0.0062	0.0044	0.0053	0.0048	

Bold values indicate the highest results obtained.

Finally, Table 8 shows the performance analysis of the RF classifier with 15 MHO techniques in terms of different TFs based on the average computational time to evaluate the impact of the MHO algorithms on the utilized dataset. Remarkably, the BA-RF method ranked first with five TFs based on $AVG_{ComputationalTime}$ (S_V1 took 19623 MS, S_V1C took 16105 MS, S_V2 took 16243 MS, S_V3 took 20211 MS, and V_V4 took 17714 MS). The AO-RF method ranked first based on S_V4 and V_V1 . The AVO-RF method ranked first based on V_V2 . Finally, the HHO-RF method ranked first based on V_V3 .

Table 8. Assessment of the impact of 9 TFs on 15 MHO algorithms with RF in terms of ($AVG_{ComputationalTime}$) (in mile seconds (MS)).

Metric	BBO	AVO	AO	SSA	ABC	PSO	BA	GWO	WOA	GOA	SFO	HHO	BSA	ASO	HGSO	Winner
<i>S_{Shaped}</i>																
S_V1 AVG	319,220	21,843	956,669	58,515	85,622	28,299	19,623	20,699	26,477	21,784	782,376	74,879	42,689	53,658	38,196	BA
STDE	72,1841	2693	5,039,076	191,113	20,898	7494	5869	2655	5256	2046	1,218,471	294,580	86,490	114,404	38,296	
S_V1C AVG	199,983	23,691	24,490	39,829	73,272	20,473	16,105	20,325	21,756	24,084	204,484	17,541	20,043	20,285	22,354	BA
STDE	445,531	3449	3311	79,666	10,469	198	933	317	545	2909	1178	649	261	234	293	
S_V2 AVG	60,455	20,426	19,027	814,391	230,972	20,915	16,243	20,704	23,881	25,771	379,603	27,815	58,313	34,136	31,841	BA
STDE	695	210	822	4,211,908	618,514	582	904	593	3936	2290	463,896	8153	158,026	8096	2876	
S_V3 AVG	124,533	25,842	23,652	38,542	169,185	23,807	20,211	78,904	59,183	22,017	467,192	269,571	425,449	23,320	33,734	BA
STDE	249,800	3275	2890	64,281	479,106	4210	3237	205,230	140,360	310	1,111,541	1,215,845	1,479,729	9674	8086	
S_V4 AVG	72,694	20,438	16,095	32,900	180,395	20,316	16,614	30,665	131,328	22,963	142,755	17,315	20,400	19,853	23,586	AO
STDE	15,501	302	837	2055	412,791	2982	1608	43,904	491,993	1571	132,943	1075	1247	361	1872	
<i>V_{Shaped}</i>																
V_V1 AVG	67,891	22,233	22,019	23,875	90,645	29,267	23,802	34,427	94,252	23,884	1,102,160	25,430	61,505	27,783	79,981	AO
STDE	9439	1892	1961	3061	58,871	4281	2838	7587	275,789	3544	3,465,191	3020	175,578	11,834	199,837	
V_V2 AVG	59,643	19,003	19,318	20,723	57,761	996,075	22,538	85,120	36,687	113,410	1,310,599	20,958	22,491	28,999	27,587	AVO
STDE	4664	289	260	151	249	3,987,941	25,247	181,022	51,720	463,868	5,396,017	3363	2270	6410	5708	
V_V3 AVG	65,061	21,289	21,744	23,428	62,978	2,269,893	18,406	131,926	25,651	160,881	2,638,876	17,667	20,508	20,645	22,698	HHO
STDE	2438	315	526	3004	845	8,022,705	2668	391,730	2099	469,009	9,852,159	600	321	380	699	
V_V4 AVG	105,240	28,343	73,367	34,158	107,950	709,976	17,714	21,336	92,967	22,817	251,302	18,828	20,853	22,990	1,556,733	BA
STDE	138,228	6919	181,029	5889	13,454	3,650,573	1704	518	253,812	954	171,343	928	395	4486	5,193,120	

Bold values indicate the highest results obtained.

In the end, The results show that the SFO-RF method achieves the best average accuracy and fitness regarding 8 of 9 TFs and the best feature size regarding 2 of 9 TFs. PSO-RF, SFO-RF, and AO-RF methods also have the best feature size for 2 of 9 TFs. For the average computational time measure, the BA-RF method achieved the best result for 5 of 9 TFs, while the AO-RF method achieved the best result for only 2 of 9 TFs.

4.5. Comparisons Based on SVM Classifier Using CCF Dataset

In this subsection, we compare the performance of the SVM classifier with 15 MHO techniques based on the average classification accuracy, the average number of selected features, the average fitness values, and the average computational time to evaluate the impact of the MHO algorithms in improving classification accuracy and choosing the most appropriate features.

Firstly, the classification accuracy is based on the original SVM classifier (before FS), and the original number of features on the CCF dataset is 0.5378. On the other hand, Table 9 shows the performance analysis of the SVM classifier with fifteen MHO techniques regarding different TFs based on the average classification accuracy to evaluate the impact of the MHO algorithms on the utilized dataset. Remarkably, AVG_{ACC} based on the SFO-SVM method ranked first for four TFs (S_V1 achieved 0.9406, S_V2 achieved 0.9412,

S_V3 achieved 0.9401, and S_V4 achieved 0.9412). The AO-SVM method ranked first with V_V1C by achieving an accuracy of 0.9328. BBO-SVM ranked first with V_V1 by achieving an accuracy of 0.9347. BBO-SVM, AVO-SVM, and AO-SVM ranked first for V_V2 by achieving an accuracy of 0.9339. BBO-SVM ranked first for V_V3 by achieving an accuracy of 0.9347. Finally, AO-SVM ranked first by achieving an accuracy of 0.9339 for S_V4 .

Table 9. Assessment of the impact of 9 TFs on 15 MHO algorithms with SVM in terms of (AVG_{ACC}).

Metric	BBO	AVO	AO	SSA	ABC	PSO	BA	GWO	WOA	GOA	SFO	HHO	BSA	ASO	HGSO	Winner
<i>S_{Shaped}</i>																
S_V1 AVG	0.9333	0.9345	0.9350	0.9347	0.9333	0.9375	0.9322	0.9339	0.9353	0.9331	0.9406	0.9333	0.9328	0.9294	0.9317	SFO
STDE	0.0021	0.0034	0.0037	0.0036	0.0021	0.0042	0.0030	0.0029	0.0039	0.0015	0.0021	0.0021	0	0.0041	0.0029	
S_V1C AVG	0.9336	0.9364	0.9328	0.9356	0.9336	0.9303	0.9322	0.9347	0.9356	0.9328	0.9336	0.9333	0.9331	0.9305	0.9311	AO
STDE	0.0025	0.0042	0	0.0040	0.0025	0.0039	0.0021	0.0036	0.0040	0	0.0025	0.0021	0.0015	0.0037	0.0034	
S_V2 AVG	0.9331	0.9356	0.9353	0.9342	0.9331	0.9364	0.9319	0.9331	0.9347	0.9328	0.9412	0.9331	0.9328	0.9305	0.9317	SFO
STDE	0.0015	0.0040	0.0039	0.0031	0.0015	0.0042	0.0040	0.0015	0.0036	0	0	0.0015	0	0.0037	0.0029	
S_V3 AVG	0.9328	0.9345	0.9353	0.9339	0.9333	0.9342	0.9319	0.9328	0.9345	0.9328	0.9401	0.9331	0.9328	0.9319	0.9311	SFO
STDE	0	0.0034	0.0039	0.0029	0.0021	0.0031	0.0033	0	0.0034	0	0.0029	0.0015	0	0.0025	0.0034	
S_V4 AVG	0.9333	0.9353	0.9342	0.9353	0.9333	0.9359	0.9331	0.9350	0.9361	0.9328	0.9412	0.9333	0.9328	0.9297	0.9314	SFO
STDE	0.0021	0.0039	0.0031	0.0039	0.0021	0.0040	0.0026	0.0037	0.0041	0.0022	0	0.0021	0	0.0040	0.0031	
<i>V_{Shaped}</i>																
V_V1 AVG	0.9347	0.9339	0.9328	0.9339	0.9328	0.9311	0.9283	0.9331	0.9325	0.9328	0.9339	0.9336	0.9331	0.9272	0.9283	BBO
STDE	0.0036	0.0029	0	0.0029	0	0.0034	0.0042	0.0015	0.0015	0	0.0029	0.0025	0.0015	0.0045	0.0042	
V_V2 AVG	0.9339	0.9339	0.9339	0.9331	0.9331	0.9308	0.9297	0.9333	0.9325	0.9328	0.9336	0.9331	0.9331	0.9272	0.9289	BBO, AVO, AO
STDE	0.0029	0.0029	0.0029	0.0015	0.0015	0.0036	0.0040	0.0021	0.0015	0	0.0025	0.0015	0.0015	0.0045	0.0042	
V_V3 AVG	0.9347	0.9331	0.9342	0.9336	0.9328	0.9322	0.9286	0.9333	0.9328	0.9328	0.9336	0.9328	0.9333	0.9272	0.9328	BBO
STDE	0.0036	0.0015	0.0031	0.0025	0	0.0021	0.0042	0.0021	0.0022	0	0.0025	0	0.0021	0.0040	0.0041	
V_V4 AVG	0.9333	0.9336	0.9339	0.9331	0.9331	0.9319	0.9297	0.9336	0.9328	0.9328	0.9333	0.9328	0.9328	0.9275	0.9258	AO
STDE	0.0021	0.0025	0.0029	0.0015	0.0015	0.0025	0.0040	0.0025	0	0	0.0021	0	0	0.0040	0.0031	

Bold values indicate the highest results obtained.

Secondly, Table 10 shows the performance analysis of the SVM classifier and 15 MHO techniques in terms of different TFs based on the average number of selected features on the utilized dataset. It is remarkable that $AVG_{Features}$ based on the AO-SVM ranked first for five TFs (S_V1 selected 1.5333 features, S_V2 selected 1.3667 features, S_V3 selected 1.5 features, V_V1 selected 4.6333 features, and 5 features were selected by V_V3). SSA-SVM ranked first for two TFs (S_V1C selected 1.9 features, and S_V4 selected 1.5333). BBO-SVM ranked first according to V_V2 by selecting 5.2 features. Finally, BBO-SVM and AVO-SVM ranked first according to V_V4 by selecting five features.

Table 10. Assessment of the impact of 9 TFs on 15 MHO algorithms with SVM in terms of ($AVG_{Features}$).

Metric	BBO	AVO	AO	SSA	ABC	PSO	BA	GWO	WOA	GOA	SFO	HHO	BSA	ASO	HGSO	Winner
<i>S_{Shaped}</i>																
S_V1 AVG	7.4667	1.8667	1.5333	1.7333	7.5000	1.9000	5.6000	4.4000	2.6333	8.5333	2.0000	7.5667	8.9667	9.7333	10.9667	AO
STDE	1.1175	1.5861	0.9568	1.6316	1.6482	0.9074	4.1280	2.3749	1.3287	1.6680	0.3651	2.4452	2.1053	2.5940	2.3872	
S_V1C AVG	6.9667	2.0000	8.2333	1.9000	7.1667	13.3000	8.9000	4.5333	2.0000	8.4333	3.1667	6.4000	8.2000	10.2000	11.1000	SSA
STDE	1.8526	1.2111	2.6036	1.1060	1.3187	2.4786	3.8153	2.1092	1.3904	1.5424	2.4911	2.2000	1.6613	2.1510	1.5351	
S_V2 AVG	7.4000	2.8667	1.3667	2.3667	7.4333	3.1667	5.1667	7.1667	4.1000	9.0333	2.4000	7.6667	8.6000	9.7667	10.9333	AO
STDE	1.0832	1.9448	0.6046	1.4020	1.5206	1.7528	3.6522	1.8273	2.9704	1.4488	0.8406	1.6600	1.2275	2.1242	1.8607	
S_V3 AVG	7.4667	3.2667	1.5000	3.4333	7.6333	4.6667	6.9000	7.2667	4.5333	8.6333	3.3000	8.3667	8.7667	9.3667	10.7333	AO
STDE	0.9214	2.2940	0.7638	2.5649	1.3287	1.5986	3.8328	1.5478	2.9970	1.3536	1.8646	1.6630	1.7065	1.5380	1.7876	
S_V4 AVG	7.6000	2.2333	1.6667	1.5333	6.6000	1.6000	3.8333	2.8667	2.0667	9.4000	2.3333	5.5000	8.4667	10.4333	11.1000	SSA
STDE	1.5406	1.7641	1.0750	0.8459	1.4283	0.8406	3.1842	1.9276	1.7114	2.2301	0.6498	2.7295	1.9956	1.6469	2.4269	
<i>V_{Shaped}</i>																
V_V1 AVG	5.1667	5.9333	4.6333	5.5667	9.2667	11.4000	10.7333	6.8000	8.7667	8.9000	7.2333	6.9667	8.1333	11.6667	11.9000	AO
STDE	1.7528	1.4591	0.8360	1.4533	1.7308	1.6653	2.4486	1.7205	2.0766	1.9209	1.5850	2.4150	2.3200	2.0221	2.3714	
V_V2 AVG	5.2000	5.8667	5.2333	5.3000	9.3000	11.2000	10.8000	6.9667	8.7333	9.1000	7.1333	6.4333	8.1000	11.6667	11.9333	BBO
STDE	1.5144	1.7461	1.6869	1.5948	1.7349	2.1664	2.3152	1.6829	1.9653	2.2561	1.5217	1.4761	1.8859	1.6799	2.3228	
V_V3 AVG	5.3333	5.4667	5.0000	5.7000	8.0333	10.7333	9.7000	6.6000	8.3667	8.3667	6.5333	6.0333	8.0333	11.2667	11.7333	AO
STDE	2.1960	1.4996	1.8797	1.3454	1.4020	1.5902	1.7349	1.8000	2.1677	1.3034	1.2037	1.5162	1.6829	2.6196	1.9653	
V_V4 AVG	5.0000	5.0000	5.0667	5.8667	8.4000	10.3667	10.8667	6.5333	8.5333	8.1667	6.9667	6.5000	7.9000	10.7667	11.4000	BBO, AVO
STDE	1.2649	1.3416	1.9653	1.8025	1.1431	2.1523	2.6675	1.7269	1.9956	1.7717	1.6630	1.7654	1.4224	2.4991	1.6042	

Bold values indicate the highest results obtained.

Thirdly, Table 11 shows the performance analysis of the SVM classifier with 15 MHO techniques in terms of different TFs based on the average classification fitness to evaluate the impact of the MHO algorithms on the utilized dataset. Remarkably, (AVG_{Fit}) based on the SFO-SVM ranked first with 4 TFs (S_V1 achieved a fitness value of 0.0595, S_V2 achieved a fitness value of 0.0591, S_V3 achieved a fitness value of 0.0605, and S_V4 achieved a fitness value of 0.0590). AVO-SVM ranked first with S_V1C by achieving a fitness value of 0.0636. BBO-SVM ranked first with V_V1 , V_V2 , and V_V3 by achieving fitness values of 0.0664, 0.0672, 0.0665 sequentially. Finally, AO-SVM ranked first by achieving a fitness value of 0.0672 for V_V4 .

Table 11. Assessment of the impact of 9 TFs on 15 MHO algorithms with SVM in terms of (AVG_{Fit}).

Metric	BBO	AVO	AO	SSA	ABC	PSO	BA	GWO	WOA	GOA	SFO	HHO	BSA	ASO	HGSO	Winner
<i>S_{Shaped}</i>																
S_V1 AVG	0.0686	0.0655	0.0649	0.0652	0.0686	0.0625	0.0690	0.0670	0.0650	0.0692	0.0595	0.0686	0.0696	0.0732	0.0714	SFO
STDE	0.0019	0.0032	0.0035	0.0031	0.0016	0.0039	0.0037	0.0031	0.0037	0.0016	0.0020	0.0022	0.0007	0.0045	0.0025	
S_V1C AVG	0.0681	0.0636	0.0694	0.0644	0.0682	0.0736	0.0702	0.0662	0.0645	0.0695	0.0668	0.0682	0.0691	0.0723	0.0720	AVO
STDE	0.0022	0.0039	0.0009	0.0037	0.0022	0.0038	0.0027	0.0038	0.0038	0.0005	0.0021	0.0023	0.0016	0.0038	0.0031	
S_V2 AVG	0.0688	0.0648	0.0645	0.0660	0.0688	0.0640	0.0692	0.0687	0.0660	0.0697	0.0591	0.0689	0.0695	0.0721	0.0714	SFO
STDE	0.0014	0.0040	0.0036	0.0029	0.0012	0.0040	0.0047	0.0013	0.0037	0.0005	0.0003	0.0016	0.0004	0.0038	0.0027	
S_V3 AVG	0.0691	0.0660	0.0646	0.0666	0.0686	0.0668	0.0698	0.0691	0.0665	0.0695	0.0605	0.0692	0.0696	0.0706	0.0719	SFO
STDE	0.0003	0.0035	0.0036	0.0027	0.0018	0.0029	0.0040	0.0005	0.0033	0.0005	0.0026	0.0017	0.0006	0.0024	0.0031	
S_V4 AVG	0.0686	0.0648	0.0657	0.0646	0.0683	0.0641	0.0676	0.0653	0.0639	0.0698	0.0590	0.0679	0.0695	0.0732	0.0718	SFO
STDE	0.0018	0.0036	0.0030	0.0036	0.0020	0.0038	0.0032	0.0036	0.0037	0.0022	0.0002	0.0024	0.0007	0.0040	0.0029	
<i>V_{Shaped}</i>																
V_V1 AVG	0.0664	0.0675	0.0682	0.0674	0.0698	0.0721	0.0747	0.0686	0.0699	0.0696	0.0679	0.0681	0.0691	0.0761	0.0751	BBO
STDE	0.0033	0.0027	0.0003	0.0027	0.0006	0.0033	0.0041	0.0013	0.0018	0.0007	0.0023	0.0023	0.0017	0.0044	0.0038	
V_V2 AVG	0.0672	0.0675	0.0672	0.0681	0.0695	0.0724	0.0733	0.0684	0.0698	0.0697	0.0682	0.0685	0.0691	0.0761	0.0746	BBO
STDE	0.0027	0.0026	0.0024	0.0014	0.0016	0.0035	0.0042	0.0022	0.0015	0.0008	0.0022	0.0017	0.0017	0.0044	0.0038	
V_V3 AVG	0.0665	0.0682	0.0669	0.0677	0.0693	0.0708	0.0741	0.0683	0.0694	0.0694	0.0680	0.0686	0.0688	0.0760	0.0756	BBO
STDE	0.0032	0.0012	0.0029	0.0025	0.0005	0.0021	0.0042	0.0021	0.0021	0.0004	0.0023	0.0005	0.0021	0.0041	0.0038	
V_V4 AVG	0.0677	0.0674	0.0672	0.0683	0.0692	0.07100	0.0734	0.0680	0.0695	0.0694	0.0684	0.0688	0.0693	0.0755	0.0774	AO
STDE	0.0019	0.0025	0.0027	0.0017	0.0014	0.0024	0.0040	0.0024	0.0007	0.0006	0.0016	0.0006	0.0005	0.0037	0.0030	

Bold values indicate the highest results obtained.

Finally, Table 12 shows the performance analysis of the SVM classifier with 15 MHO techniques regarding different TFs based on the average computational time to evaluate the impact of the MHO algorithms on the utilized dataset.

Remarkably, $AVG_ComputationalTime$ with the HHO-SVM method ranked first in five TFs (S_V2 took 9258, V_V1 took 6164 MS, V_V2 took 6059 MS, V_V3 took 6221 MS, and V_V4 took 5757MS). PSO-SVM ranked first with S_V1 by taking 6395 MS and 5365 MS with S_V4. WOA-SVM ranked first with S_V1C by taking 7940 MS. AVO-SVM method ranked first with V_V3 by taking 8993 MS. Finally, PSO-SVM method ranked first based on S_V4 by taking 5365 MS.

Table 12. Assessment of the impact of 9 TFs on 15 MHO algorithms based on SVM in terms of average computational time (in mile seconds (MS)).

Metric	BBO	AVO	AO	SSA	ABC	PSO	BA	GWO	WOA	GOA	SFO	HHO	BSA	ASO	HGSO	Winner
<i>S_{Shaped}</i>																
S_V1 AVG	28,979	7576	6793	9132	29,619	6395	8650	7793	8040	10,634	68,893	8866	9173	8362	11,120	PSO
STDE	1615	872	464	714	1537	571	1331	963	942	741	1670	445	1181	1407	382	
S_V1C AVG	28,961	8352	12,587	9164	30,123	12,708	8445	8008.9667	7940	11,275	685,099	30,220	13,432	9856	17,557	WOA
STDE	1777	560	304	756	1581	1105	1174	1074	1249	1028	2,896,827	97,500	4872	1680	7035	
S_V2 AVG	44,779	11,951	10,346	12,562	43,858	13,723	113,592	16,109	11,862	15,043	316,007	9258	10,046	124,445	12,331	HHO
STDE	6882	2170	1373	2916	14634	4637	513,405	28,643	2444	3179	637,806	526	855	612,622	1657	
S_V3 AVG	31,652	8993	9466	11,838	36,429	10,258	57,083	11,038	163,721	14,692	1,257,449	13,233	12,757	10,340	14,005	AVO
STDE	1285	801	1362	3408	6745	2594	180,428	1223	410,106	2976	5,486,947	1293	620	1871	546	
S_V4 AVG	30,523	7786	6795	9617	28,089	5365	9132	7588	7969	10,657	51,322	9302	8565	8776	11,442	PSO
STDE	4659	1545	683	683	2462	1043	1286	1268	964	1759	1286	554	1522	1861	451	
<i>V_{Shaped}</i>																
V_V1 AVG	25,897	8207	10,044	11,464	615,261	12,892	11,086	8892	11,389	12,154	129,525	6164	9705	543,876	11,687	HHO
STDE	2061	1000	602	636	2,204,345	1734	963	1736	1719	1585	2181	7534	1831	2,829,738	309	
V_V2 AVG	26,008	9518	9736	11,033	34,844	12,846	10,996	8852	11,269	12,179	185,319	6059	9739	14,900	11,669	HHO
STDE	2304	2655	423	643	2263	1777	769	1804	1723	1651	277,300	499	1830	629	361	
V_V3 AVG	77,736	8405	197,424	23,817	93,181	26,760	17,351	13,852	11,466	14,751	272,414	6221	21,470	17,763	11,517	HHO
STDE	156,763	1227	573,179	11,233	38,736	13,692	5253	14,535	1959	3924	659,991	967	46,755	3137	353	
V_V4 AVG	317,211	596,142	17,783	10,940	187,136	12,477	11,095	8223	11,168	11,388	929,099	5757	9776	14,575	11,274	HHO
STDE	893,841	2,389,683	30,533	729	808,156	1691	666	1555	1752	1162	3,428,264	459	1359	567	325	

Bold values indicate the highest results obtained.

In the end, the results show that the SFO-SVM method achieves the best average accuracy and fitness for 4 of 9 TFs sequentially. At the same time, BBO-SVM and AO-SVM methods achieve the best accuracy for 3 of 9 TFs. For average feature size, AO-SVM performs well for 5 of 9, and BBO-SVM and SSA-SVM perform well for 2 of 9 TFs. For average fitness, BBO-SVM performs well for 3 of 9 TFs. HHO-SVM reaches the best result in computational time measure for 5 of 9 TFs, while the PSO-SVM method achieves the best result for only 2 of 9 TFs.

Finally, the SFO method achieves the best average accuracy and fitness results for two classifiers, RF and SVM, and with most TFs.

4.6. Comparing with Other Studies That Utilized the Credit European Cardholders Dataset

In this subsection, a detailed comparison of various research efforts on the same CCF Kaggle dataset is presented. Table 13 illustrates the impact of different methodologies on classification accuracy. Notably, our research (CCFD) distinguishes itself by applying the MHO (SFO) feature selection technique, achieving accuracy rates of 97.79% with the RF classifier and 94.12% with the support SVM classifier, employing under-sampling to

mitigate data imbalance. This improvement over earlier studies, such as the 2018 research by Lakshmi and Selvani, achieving 95.50% accuracy using RF and oversampling, and the 2023 studies by Mniai et al., who utilized varied feature selection methods but attained lower accuracies, highlights the effectiveness of MHO (SFO). Our findings suggest that advanced feature selection techniques like MHO (SFO) can significantly boost machine learning performance, providing a robust approach for managing intricate datasets.

Table 13. Accuracy of different research for CCF dataset.

Year	Author	Accuracy	Methodology	Feature Selection Technique	Sampling Technique
2018	Lakshmi and Selvani [83]	0.9550	RF	-	Oversampling
2023	Almazroi and Ayub [84]	0.5600	SVM	-	SMOTE
2023	Mniai et. al. [82]	0.8900	RF	-	Undersampling
2023	Mniai et. al. [82]	0.8300	SVM	-	Undersampling
2023	Mniai et. al. [82]	0.9100	RF	Filter (Rank Order Filter)	Undersampling
2023	Mniai et. al. [82]	0.8300	SVM	Filter (Rank Order Filter)	Undersampling
2023	Mniai et. al. [82]	0.8800	RF	Wrapper (Recursive Feature Elimination)	Undersampling
2023	Mniai et. al. [82]	0.8200	SVM	Wrapper (Recursive Feature Elimination)	Undersampling
2023	Mniai et. al. [82]	0.8900	RF	Embedded	Undersampling
2023	Mniai et. al. [82]	0.8300	SVM	Embedded	Undersampling
2024	CCFD (Our Research)	0.9779	RF	MHO (SFO)	Undersampling
2024	CCFD (Our Research)	0.9412	SVM	MHO (SFO)	Undersampling

Bold values indicate the highest results obtained.

4.7. Convergence Investigation

This section examines the performance of 4 methods with 15 MHO techniques (RF (with S_{Shaped} TFs), RF (with V_{Shaped} TFs), SVM (with S_{Shaped} TFs), and SVM (with V_{Shaped} TFs)) for handling the FS strategy using the European credit-cardholders dataset. The aim is to evaluate their convergence capabilities, as shown in figures sequentially labeled Figures 4–7. These graphs indicate that the SFO-RF algorithm demonstrated superior and optimal convergence over the dataset compared to its peers, assessed under the same population size and iteration number conditions.

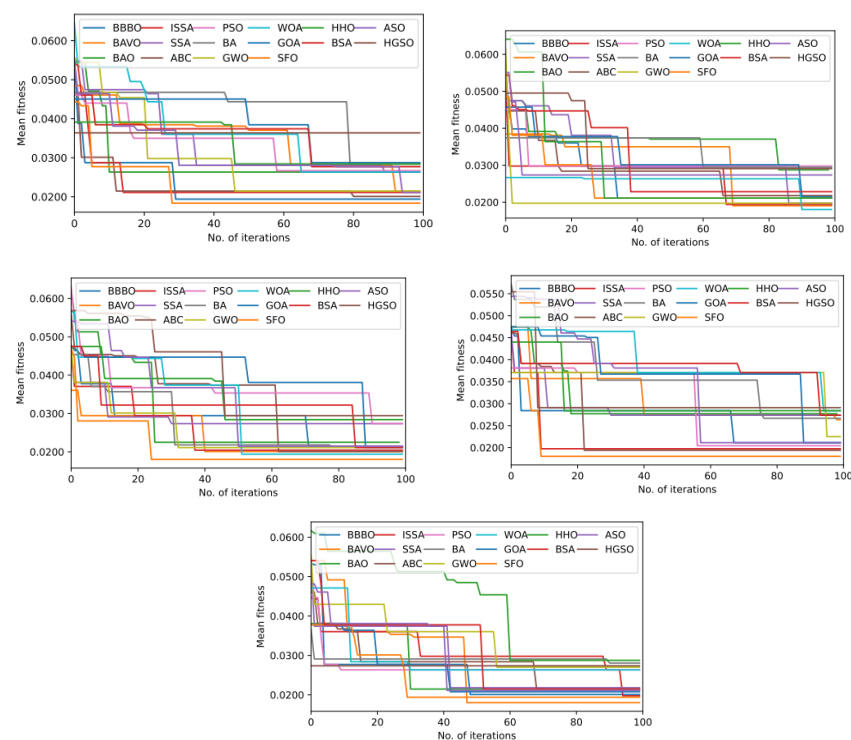


Figure 4. Convergence curve of 15 MHO techniques based on the RF classifier with S_{shaped} .

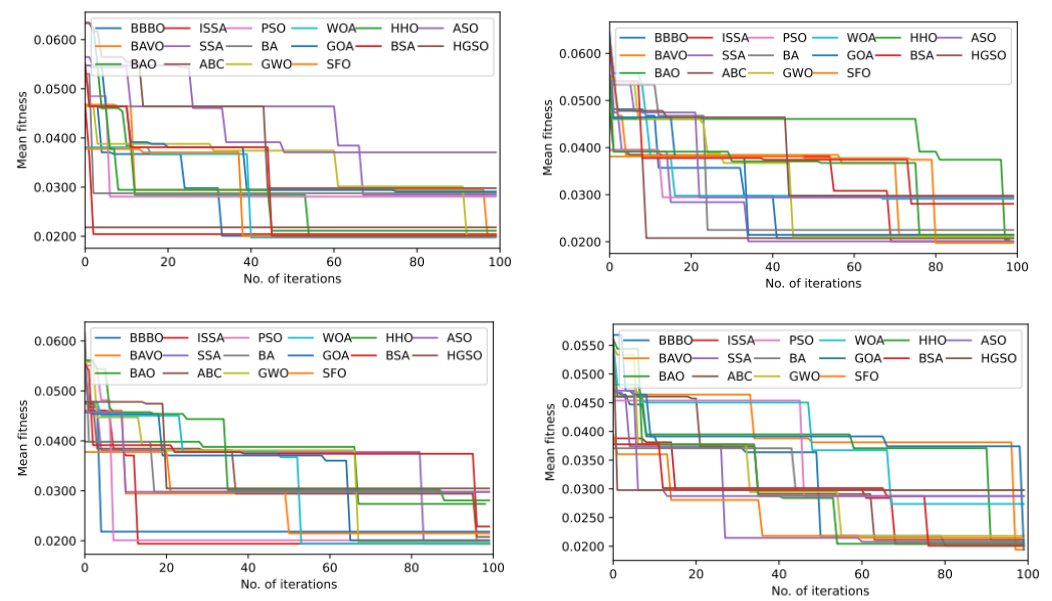


Figure 5. Convergence curve of 15 MHO techniques based on the RF classifier with *V-shaped*.

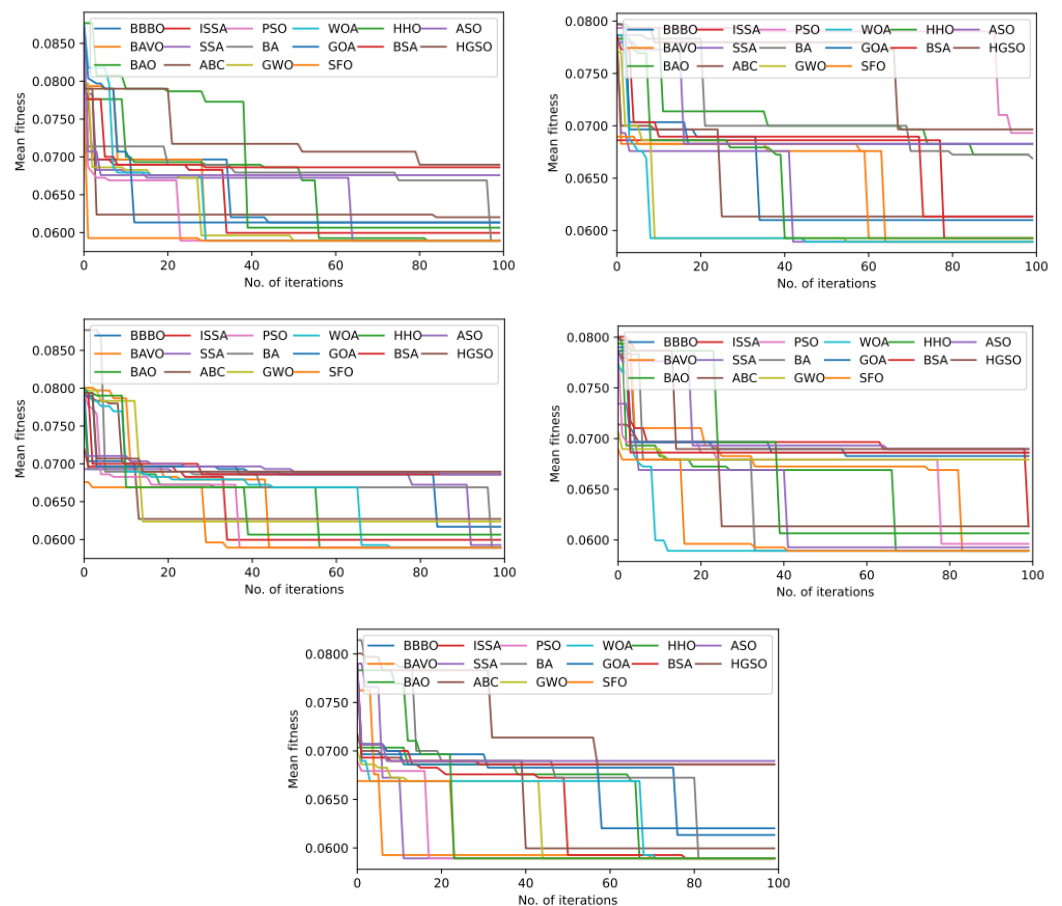


Figure 6. Convergence curve of 15 MHO techniques based on the SVM classifier with *S-shaped*.

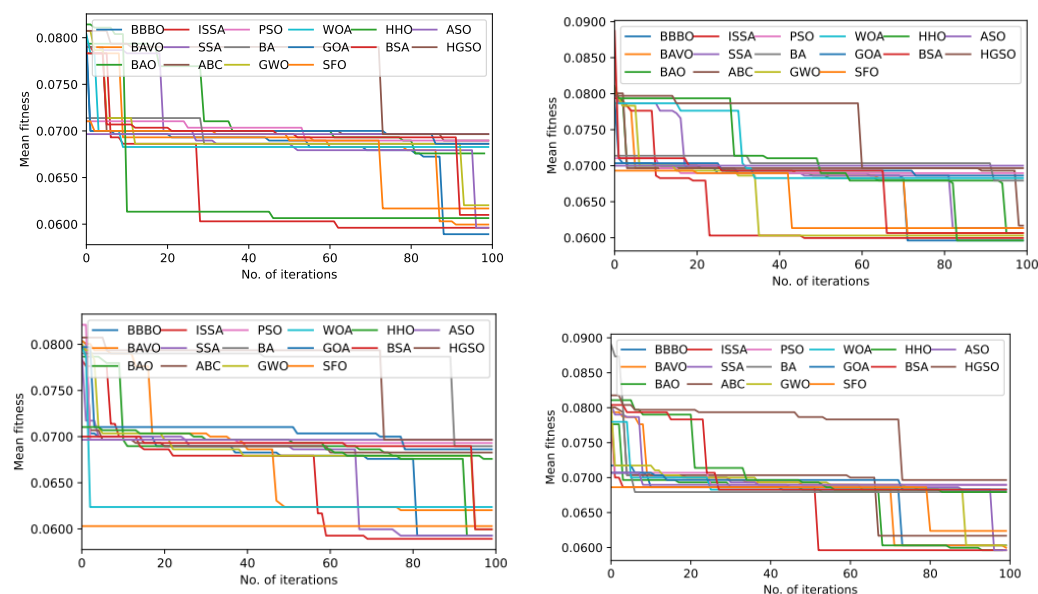


Figure 7. Convergence curve of 15 MHO techniques based on the SVM classifier with V -shaped.

4.8. Evaluating Model Robustness and Generalization

Detecting fraud in financial transactions is a critical task that demands reliable and adaptable predictive models. In this study, we assessed the robustness and ability to generalize of our chosen model for fraud detection. After comprehensive experimentation, we selected the best-performing model from our experiments in term of accuracy, which was trained on a balanced dataset. The chosen model utilized a random forest classifier, along with a V_{V1} shaped transfer function and sailfish optimizer. To evaluate its effectiveness, we tested the model on an imbalanced dataset comprising 20% of the original data, totaling 284,807 samples. The model achieved an accuracy of 97.14% on this imbalanced test set, demonstrating its robustness and ability to generalize effectively to real-world data distributions.

5. Conclusions

This study addressed the significant challenge of data imbalance, which significantly impedes the efficiency of CCFD models, by creating a model that enhances CC security and supports the advancement of E-commerce. The model is structured into four phases: data preprocessing, FS, applying ML classifiers, and evaluating these classifiers. The dataset was collected from a Kaggle repository, and the optimal feature set was selected using 15 MHO algorithms employing 9 TFs. Two ML classifiers, RF and SVM, were employed to compare the outcomes on the whole and selected features. The results revealed an improvement in accuracy by 4% for RF and 41% for SVM when using the selected features over the full set of features. SFO-RF with V_{V1} shaped ranked first regarding classification accuracy by achieving 97.79%. AO-SVM with S_{V3} shaped selected the least number of features (1.5) while obtaining an accuracy of 93.53%. SFO-RF with S_{V2} shaped ranked first regarding classification error by a fitness value of 0.0248%. PSO-SVM with S_{V4} shaped ranked first regarding computational time by recording 5365 MS. Therefore, SFO-RF is the overall best method by achieving the greatest classification accuracy with V_{V1} and least fitness value with S_{V2} . The future research should explore integrating ML with various MHO techniques and additional classifiers, such as deep learning, to further validate the efficiency of MHO methods in feature selection for classification tasks. They are, additionally, exploring hybrid models that combine the strengths of different techniques, such as integrating under-sampling with over-sampling strategies like SMOTE. Hybrid models offer a promising avenue to enhance the robustness and performance of FDSs further. By leveraging the complementary nature of various sampling and modeling techniques, we anticipate achieving improved

accuracy in identifying fraudulent transactions while maintaining computational efficiency and interpretability.

Author Contributions: A.A.A. conceived the idea for the CCFD method, designed the experimental framework, and analyzed the results. F.A.M. contributed to designing and implementing the meta-heuristic optimization techniques and data analysis, providing insights into feature selection. D.T.M. worked on the technical aspects of data processing, assisted in interpreting the data, and provided critical feedback on the methodology. S.E.S. participated in the design and execution of the study and contributed to the critical review and editing of the manuscript. All authors collectively contributed to the writing and revision of the manuscript, provided critical feedback on the methodology, and approved the final version of the paper for submission. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Deanship of Scientific Research, Vice Presidency for Graduate Studies and Scientific Research, King Faisal University, Saudi Arabia, under Project Grant KFU241160.

Data Availability Statement: The data and code used in this study can be found at <https://github.com/FMaghraby/CCFD> (accessed on 16 July 2024).

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

BBO	Brown-bear optimization
AVO	African vultures optimization
AO	Aquila optimization
SSA	Sparrow search algorithm
ABC	Artificial bee colony
PSO	Particle swarm optimization
BA	Bat algorithm
GWO	Grey wolf optimization
WOA	Whale optimization algorithm
GOA	Grasshopper optimization algorithm
SFO	Sailfish optimizer
HHO	Harris hawks optimization
BSA	Bird swarm algorithm
ASO	Atom search optimization
HGSO	Henry gas solubility optimization
RF	Random forest
SVM	Support vector machine
TFs	Transfer functions
AVG_{ACC}	Average accuracy
STDE	Standard deviation
CCF	Credit card fraud
DM	Data mining
DL	Deep learning
CCFD	Credit card fraud detection
FD	Fraud detection
CC	Credit card
FDSs	Fraud detection systems
FS	Feature selection
MHOA	Meta-heuristic optimization Algorithm
ML	Machine learning

References

1. Song, Y.; Escobar, O.; Arzubuaga, U.; De Massis, A. The digital transformation of a traditional market into an entrepreneurial ecosystem. *Rev. Manag. Sci.* **2022**, *16*, 65–88. [[CrossRef](#)]
2. Lucas, Y.; Jurgovsky, J. Credit card fraud detection using machine learning: A survey. *arXiv* **2020**, arXiv:2010.06479.

3. Liu, Y.; Gao, W.; Hua, R.; Chen, H. Decomposition and measurement of economic effects of E-commerce based on static feder model and improved dynamic feder model. In Proceedings of the 2021 2nd International Conference on E-Commerce and Internet Technology (ECIT), Hangzhou, China, 5–7 March 2021; pp. 213–217.
4. Tran, L.T.T. Managing the effectiveness of e-commerce platforms in a pandemic. *J. Retail. Consum. Serv.* **2021**, *58*, 102287. [\[CrossRef\]](#)
5. Laudon, K.C.; Laudon, J.P. *Management Information Systems: Managing the Digital Firm*, 17th ed.; Pearson Educación: London, UK, 2023.
6. Fanai, H.; Abbasimehr, H. A novel combined approach based on deep Autoencoder and deep classifiers for credit card fraud detection. *Expert Syst. Appl.* **2023**, *217*, 119562. [\[CrossRef\]](#)
7. Singh, A.; Jain, A.; Biabale, S.E. Financial Fraud Detection Approach Based on Firefly Optimization Algorithm and Support Vector Machine. *Appl. Comput. Intell. Soft Comput.* **2022**, *2022*, 1468015. [\[CrossRef\]](#)
8. Wahid, A.; Msahli, M.; Bifet, A.; Memmi, G. NFA: A neural factorization autoencoder based online telephony fraud detection. *Digit. Commun. Netw.* **2023**, *10*, 158–167. [\[CrossRef\]](#)
9. Carta, S.; Fenu, G.; Recupero, D.R.; Saia, R. Fraud detection for E-commerce transactions by employing a prudential Multiple Consensus model. *J. Inf. Secur. Appl.* **2019**, *46*, 13–22. [\[CrossRef\]](#)
10. Rodrigues, V.F.; Policarpo, L.M.; da Silveira, D.E.; da Rosa Righi, R.; da Costa, C.A.; Barbosa, J.L.V.; Antunes, R.S.; Scorsatto, R.; Arcot, T. Fraud detection and prevention in e-commerce: A systematic literature review. *Electron. Commer. Res. Appl.* **2022**, *7*, 101207. [\[CrossRef\]](#)
11. Alamri, M.; Ykhlef, M. Survey of Credit Card Anomaly and Fraud Detection Using Sampling Techniques. *Electronics* **2022**, *11*, 4003. [\[CrossRef\]](#)
12. Asha, R.; KR, S.K. Credit card fraud detection using artificial neural network. *Glob. Transit. Proc.* **2021**, *2*, 35–41.
13. Bin Sulaiman, R.; Schetinin, V.; Sant, P. Review of machine learning approach on credit card fraud detection. *Hum.-Centric Intell. Syst.* **2022**, *2*, 55–68. [\[CrossRef\]](#)
14. Bao, Y.; Hilary, G.; Ke, B. Artificial intelligence and fraud detection. In *Innovative Technology at the Interface of Finance and Operations*; Springer: Berlin/Heidelberg, Germany, 2022; Volume I, pp. 223–247.
15. Nandi, A.K.; Randhawa, K.K.; Chua, H.S.; Seera, M.; Lim, C.P. Credit card fraud detection using a hierarchical behavior-knowledge space model. *PLoS ONE* **2022**, *17*, e0260579. [\[CrossRef\]](#) [\[PubMed\]](#)
16. Agarwal, A.; Ratha, N.K. Black-Box Adversarial Entry in Finance through Credit Card Fraud Detection. In Proceedings of the CIKM Workshops, Gold Coast, QLD, Australia, 1–5 November 2021.
17. Faris, H.; Mafarja, M.M.; Heidari, A.A.; Aljarah, I.; Ala'm, A.Z.; Mirjalili, S.; Fujita, H. An efficient binary salp swarm algorithm with crossover scheme for feature selection problems. *Knowl.-Based Syst.* **2018**, *154*, 43–67. [\[CrossRef\]](#)
18. Prakash, T.; Singh, P.P.; Singh, V.P.; Singh, S.N. A Novel Brown-bear Optimization Algorithm for Solving Economic Dispatch Problem. In *Advanced Control & Optimization Paradigms for Energy System Operation and Management*; River Publishers: Aalborg, Denmark, 2023; pp. 137–164.
19. Cartella, F.; Anunciacao, O.; Funabiki, Y.; Yamaguchi, D.; Akishita, T.; Elshocht, O. Adversarial attacks for tabular data: Application to fraud detection and imbalanced data. *arXiv* **2021**, arXiv:2101.08030.
20. Beheshti, Z.; Shamsuddin, S.M.H. A review of population-based meta-heuristic algorithms. *Int. J. Adv. Soft Comput. Appl.* **2013**, *5*, 18298676.
21. Agrawal, P.; Abutarboush, H.F.; Ganesh, T.; Mohamed, A.W. Metaheuristic algorithms on feature selection: A survey of one decade of research (2009–2019). *IEEE Access* **2021**, *9*, 26766–26791. [\[CrossRef\]](#)
22. Abualigah, L.; Diabat, A.; Geem, Z.W. A comprehensive survey of the harmony search algorithm in clustering applications. *Appl. Sci.* **2020**, *10*, 3827. [\[CrossRef\]](#)
23. Salcedo-Sanz, S. Modern meta-heuristics based on nonlinear physics processes: A review of models and design procedures. *Phys. Rep.* **2016**, *655*, 1–70. [\[CrossRef\]](#)
24. Palimkar, P.; Shaw, R.N.; Ghosh, A. Machine learning technique to prognosis diabetes disease: Random forest classifier approach. In *Proceedings of the Advanced Computing and Intelligent Technologies: Proceedings of ICACIT 2021*; Springer: Berlin/Heidelberg, Germany, 2022; pp. 219–244.
25. Phan, T.N.; Kuch, V.; Lehnert, L.W. Land cover classification using Google Earth Engine and random forest classifier—The role of image composition. *Remote Sens.* **2020**, *12*, 2411. [\[CrossRef\]](#)
26. Pisser, D.A.; Schnyer, D.M. Support vector machine. In *Machine Learning*; Elsevier: Amsterdam, The Netherlands, 2020; pp. 101–121.
27. Zojaji, Z.; Atani, R.E.; Monadjemi, A.H. A survey of credit card fraud detection techniques: Data and technique oriented perspective. *arXiv* **2016**, arXiv:1611.06439.
28. Adewumi, A.O.; Akinyelu, A.A. A survey of machine-learning and nature-inspired based credit card fraud detection techniques. *Int. J. Syst. Assur. Eng. Manag.* **2017**, *8*, 937–953. [\[CrossRef\]](#)
29. Chilaka, U.; Chukwudebe, G.; Bashiru, A. A review of credit card fraud detection techniques in electronic finance and banking. *Conic Res. Eng. J.* **2019**, *3*, 456–467.
30. Khalid, A.R.; Owoh, N.; Uthmani, O.; Ashawa, M.; Osamor, J.; Adejoh, J. Enhancing credit card fraud detection: An ensemble machine learning approach. *Big Data Cogn. Comput.* **2024**, *8*, 6. [\[CrossRef\]](#)

31. Abdul Salam, M.; Fouad, K.M.; Elbably, D.L.; Elsayed, S.M. Federated learning model for credit card fraud detection with data balancing techniques. *Neural Comput. Appl.* **2024**, *36*, 6231–6256. [\[CrossRef\]](#)
32. Chen, C.T.; Lee, C.; Huang, S.H.; Peng, W.C. Credit Card Fraud Detection via Intelligent Sampling and Self-supervised Learning. *Acm Trans. Intell. Syst. Technol.* **2024**, *15*, 1–29. [\[CrossRef\]](#)
33. Taha, A.A.; Malebary, S.J. An intelligent approach to credit card fraud detection using an optimized light gradient boosting machine. *IEEE Access* **2020**, *8*, 25579–25587. [\[CrossRef\]](#)
34. Rawashdeh, E.; Al-Ramahi, N.; Ahmad, H.; Zaghloul, R. Efficient credit card fraud detection using evolutionary hybrid feature selection and random weight networks. *Int. J. Data Netw. Sci.* **2024**, *8*, 463–472. [\[CrossRef\]](#)
35. Kennedy, R.K.; Villanustre, F.; Khoshgoftaar, T.M.; Salekshahrezaee, Z. Synthesizing class labels for highly imbalanced credit card fraud detection data. *J. Big Data* **2024**, *11*, 38. [\[CrossRef\]](#)
36. Aziz, A.; Ghous, H. Fraudulent transactions detection in credit card by using data mining methods: A review. *Int. J. Sci. Prog. Res. (IJSPR)* **2021**, *79*, 31–48.
37. Nguyen, T.T.; Tahir, H.; Abdelrazek, M.; Babar, A. Deep learning methods for credit card fraud detection. *arXiv* **2020**, arXiv:2012.03754.
38. Ahmad, I. Feature selection using particle swarm optimization in intrusion detection. *Int. J. Distrib. Sens. Netw.* **2015**, *11*, 806954. [\[CrossRef\]](#)
39. Rtayli, N.; Enneya, N. Selection features and support vector machine for credit card risk identification. *Procedia Manuf.* **2020**, *46*, 941–948. [\[CrossRef\]](#)
40. Misra, S.; Thakur, S.; Ghosh, M.; Saha, S.K. An autoencoder based model for detecting fraudulent credit card transaction. *Procedia Comput. Sci.* **2020**, *167*, 254–262. [\[CrossRef\]](#)
41. Schlör, D.; Ring, M.; Krause, A.; Hotho, A. Financial fraud detection with improved neural arithmetic logic units. In Proceedings of the Mining Data for Financial Applications: 5th ECML PKDD Workshop, MIDAS 2020, Ghent, Belgium, 18 September 2020; Revised Selected Papers 5. Springer: Berlin/Heidelberg, Germany, 2021; pp. 40–54.
42. Buschjäger, S.; Honysz, P.J.; Morik, K. Randomized outlier detection with trees. *Int. J. Data Sci. Anal.* **2022**, *13*, 91–104. [\[CrossRef\]](#)
43. Hajek, P.; Abedin, M.Z.; Sivarajah, U. Fraud detection in mobile payment systems using an XGBoost-based framework. *Inf. Syst. Front.* **2022**, *25*, 1985–2003. [\[CrossRef\]](#) [\[PubMed\]](#)
44. Kim, J.; Kim, H.J.; Kim, H. Fraud detection for job placement using hierarchical clusters-based deep neural networks. *Appl. Intell.* **2019**, *49*, 2842–2861. [\[CrossRef\]](#)
45. Abdollahzadeh, B.; Gharehchopogh, F.S.; Mirjalili, S. African vultures optimization algorithm: A new nature-inspired metaheuristic algorithm for global optimization problems. *Comput. Ind. Eng.* **2021**, *158*, 107408. [\[CrossRef\]](#)
46. Abuelrub, A.; Awwad, B. An improved binary African vultures optimization approach to solve the UC problem for power systems. *Results Eng.* **2023**, *19*, 101354. [\[CrossRef\]](#)
47. Abualigah, L.; Yousri, D.; Abd Elaziz, M.; Ewees, A.A.; Al-Qaness, M.A.; Gandomi, A.H. Aquila optimizer: A novel meta-heuristic optimization algorithm. *Comput. Ind. Eng.* **2021**, *157*, 107250. [\[CrossRef\]](#)
48. Xue, J.; Shen, B. A novel swarm intelligence optimization approach: Sparrow search algorithm. *Syst. Sci. Control Eng.* **2020**, *8*, 22–34. [\[CrossRef\]](#)
49. Gad, A.G.; Sallam, K.M.; Chakraborty, R.K.; Ryan, M.J.; Abohany, A.A. An improved binary sparrow search algorithm for feature selection in data classification. *Neural Comput. Appl.* **2022**, *34*, 15705–15752. [\[CrossRef\]](#)
50. Karaboga, D. *An Idea Based on Honey Bee Swarm for Numerical Optimization*; Technical Report, Technical Report-tr06; Erciyes University, Engineering Faculty, Computer: Kayseri, Turkey, 2005.
51. Li, P.; Zhang, Y.; Gu, J.; Duan, S. Prediction of compressive strength of concrete based on improved artificial bee colony-multilayer perceptron algorithm. *Sci. Rep.* **2024**, *14*, 6414. [\[CrossRef\]](#) [\[PubMed\]](#)
52. Eberhart, R.; Kennedy, J. A new optimizer using particle swarm theory. In Proceedings of the Sixth International Symposium on Micro Machine and Human Science, MHS'95, Nagoya, Japan, 4–6 October 1995; pp. 39–43.
53. Gad, A.G. Particle swarm optimization algorithm and its applications: A systematic review. *Arch. Comput. Methods Eng.* **2022**, *29*, 2531–2561. [\[CrossRef\]](#)
54. Yang, X.S. A new metaheuristic bat-inspired algorithm. In *Nature Inspired Cooperative Strategies for Optimization (NICSO 2010)*; Springer: Berlin/Heidelberg, Germany, 2010; pp. 65–74.
55. Agarwal, T.; Kumar, V. A systematic review on bat algorithm: Theoretical foundation, variants, and applications. *Arch. Comput. Methods Eng.* **2021**, *29*, 2707–2736. [\[CrossRef\]](#)
56. Zorarpacı, E.; Özel, S.A. A hybrid approach of differential evolution and artificial bee colony for feature selection. *Expert Syst. Appl.* **2016**, *62*, 91–103. [\[CrossRef\]](#)
57. Sharma, I.; Kumar, V.; Sharma, S. A comprehensive survey on grey wolf optimization. *Recent Adv. Comput. Sci. Commun.* **2022**, *15*, 323–333.
58. Mirjalili, S.; Lewis, A. The whale optimization algorithm. *Adv. Eng. Softw.* **2016**, *95*, 51–67. [\[CrossRef\]](#)
59. Rana, N.; Latiff, M.S.A.; Abdulhamid, S.M.; Chiroma, H. Whale optimization algorithm: A systematic review of contemporary applications, modifications and developments. *Neural Comput. Appl.* **2020**, *32*, 16245–16277. [\[CrossRef\]](#)
60. Saremi, S.; Mirjalili, S.; Lewis, A. Grasshopper optimisation algorithm: Theory and application. *Adv. Eng. Softw.* **2017**, *105*, 30–47. [\[CrossRef\]](#)

61. Meraihi, Y.; Gabis, A.B.; Mirjalili, S.; Ramdane-Cherif, A. Grasshopper optimization algorithm: Theory, variants, and applications. *IEEE Access* **2021**, *9*, 50001–50024. [\[CrossRef\]](#)
62. Shadravan, S.; Naji, H.R.; Bardsiri, V.K. The Sailfish Optimizer: A novel nature-inspired metaheuristic algorithm for solving constrained engineering optimization problems. *Eng. Appl. Artif. Intell.* **2019**, *80*, 20–34. [\[CrossRef\]](#)
63. Ghosh, K.K.; Ahmed, S.; Singh, P.K.; Geem, Z.W.; Sarkar, R. Improved binary sailfish optimizer based on adaptive β -hill climbing for feature selection. *IEEE Access* **2020**, *8*, 83548–83560. [\[CrossRef\]](#)
64. Heidari, A.A.; Mirjalili, S.; Faris, H.; Aljarah, I.; Mafarja, M.; Chen, H. Harris hawks optimization: Algorithm and applications. *Future Gener. Comput. Syst.* **2019**, *97*, 849–872. [\[CrossRef\]](#)
65. Alabool, H.M.; Alarabiat, D.; Abualigah, L.; Heidari, A.A. Harris hawks optimization: A comprehensive review of recent variants and applications. *Neural Comput. Appl.* **2021**, *33*, 8939–8980. [\[CrossRef\]](#)
66. Meng, X.B.; Gao, X.Z.; Lu, L.; Liu, Y.; Zhang, H. A new bio-inspired optimisation algorithm: Bird Swarm Algorithm. *J. Exp. Theor. Artif. Intell.* **2016**, *28*, 673–687. [\[CrossRef\]](#)
67. Varol Altay, E.; Alatas, B. Bird swarm algorithms with chaotic mapping. *Artif. Intell. Rev.* **2020**, *53*, 1373–1414. [\[CrossRef\]](#)
68. Zhao, W.; Wang, L.; Zhang, Z. Atom search optimization and its application to solve a hydrogeologic parameter estimation problem. *Knowl.-Based Syst.* **2019**, *163*, 283–304. [\[CrossRef\]](#)
69. Hashim, F.A.; Houssein, E.H.; Mabrouk, M.S.; Al-Atabany, W.; Mirjalili, S. Henry gas solubility optimization: A novel physics-based algorithm. *Future Gener. Comput. Syst.* **2019**, *101*, 646–667. [\[CrossRef\]](#)
70. Mosa, D.T.; Mahmoud, A.; Zaki, J.; Sorour, S.E.; El-Sappagh, S.; Abuhmed, T. Henry gas solubility optimization double machine learning classifier for neurosurgical patients. *PLoS ONE* **2023**, *18*, e0285455. [\[CrossRef\]](#)
71. Hussien, R.M.; Abohany, A.A.; Moustafa, N.; Sallam, K.M. An improved Henry gas optimization algorithm for joint mining decision and resource allocation in a MEC-enabled blockchain networks. *Neural Comput. Appl.* **2023**, *35*, 18665–18680. [\[CrossRef\]](#)
72. Zaki, M.J.; Meira, W. *Data Mining and Analysis: Fundamental Concepts and Algorithms*; Cambridge University Press: Cambridge, UK, 2014.
73. Xiong, Z.; Sun, X.; Sang, J.; Wei, X. Modify the accuracy of MODIS PWV in China: A performance comparison using random forest, generalized regression neural network and back-propagation neural network. *Remote Sens.* **2021**, *13*, 2215. [\[CrossRef\]](#)
74. Zhang, L.; Li, X.; Zheng, D.; Zhang, K.; Ma, Q.; Zhao, Y.; Ge, Y. Merging multiple satellite-based precipitation products and gauge observations using a novel double machine learning approach. *J. Hydrol.* **2021**, *594*, 125969. [\[CrossRef\]](#)
75. Sadorsky, P. A random forests approach to predicting clean energy stock prices. *J. Risk Financ. Manag.* **2021**, *14*, 48. [\[CrossRef\]](#)
76. Vapnik, V. *The Nature of Statistical Learning Theory*; Springer Science & Business Media: Berlin/Heidelberg, Germany, 2013.
77. Huang, W.; Liu, H.; Zhang, Y.; Mi, R.; Tong, C.; Xiao, W.; Shuai, B. Railway dangerous goods transportation system risk identification: Comparisons among SVM, PSO-SVM, GA-SVM and GS-SVM. *Appl. Soft Comput.* **2021**, *109*, 107541. [\[CrossRef\]](#)
78. Ding, C.; Bao, T.Y.; Huang, H.L. Quantum-inspired support vector machine. *IEEE Trans. Neural Netw. Learn. Syst.* **2021**, *33*, 7210–7222. [\[CrossRef\]](#)
79. Mirjalili, S.; Lewis, A. S-shaped versus V-shaped arXiv:2010.06479 for binary particle swarm optimization. *Swarm Evol. Comput.* **2013**, *9*, 1–14.
80. Kennedy, J.; Eberhart, R.C. A discrete binary version of the particle swarm algorithm. In Proceedings of the 1997 IEEE International Conference on Systems, Man, and Cybernetics, Computational Cybernetics and Simulation, Orlando, FL, USA, 12–15 October 1997; Volume 5, pp. 4104–4108.
81. Rashedi, E.; Nezamabadi-Pour, H.; Saryazdi, S. BGSA: Binary gravitational search algorithm. *Nat. Comput.* **2010**, *9*, 727–745. [\[CrossRef\]](#)
82. Mnai, A.; Tarik, M.; Jebbari, K. A Novel Framework for Credit Card Fraud Detection. *IEEE Access* **2023**, *99*, 112776–112786. [\[CrossRef\]](#)
83. Lakshmi, S.; Kavilla, S.D. Machine learning for credit card fraud detection system. *Int. J. Appl. Eng. Res.* **2018**, *13*, 16819–16824.
84. Almazroi, A.A.; Ayub, N. Online Payment Fraud Detection Model Using Machine Learning Techniques. *IEEE Access* **2023**, *11*, 137188–137203. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.